

SISTEMA DE INFERENCIA DE VERTIDOS EN RÍOS

Autores:

- Duo Yu Xe
- Tadeo Adrián Troncoso Taraborreli
- Víctor Velayos Velayos

Project ID: 48d5edf4-15aa-4c42-
b2fb-09df0163cf19

Enero, 2026

Universidad Europea de Madrid, C/Tajo, s/n, Villaviciosa de Odón, Madrid 28670,
España

Proyecto de Computación I - Álvaro Manuel Rodríguez Rodríguez

Tabla de Contenidos

1. Introducción y configuración del entorno

1.1 Contexto del proyecto y motivación

1.2 Fallos “flatline” en series temporales: por qué importan

1.3 Enfoque metodológico (visión de alto nivel)

1.3.1 Detección y reparación de flatlines

1.3.2 Pipeline de preprocesamiento y serialización (de crudo a listo para análisis)

1.4 Configuración inicial del entorno de análisis (stack común)

1.5 Configuración del entorno para procesamiento acelerado por GPU

1.5.1 Librerías del pipeline y rol de cada componente

1.5.2 Rutas del proyecto y limpieza segura de artefactos

1.5.3 Verificación de GPU antes de cargar librerías aceleradas

2. Carga de datos y exploración inicial

2.1 Carga de datos y exploración inicial

2.1.1 Objetivo transversal: esquema, tipado y completitud

2.2 Series temporales de sensores: de JSON a tabla homogénea

2.2.1 Parseo y normalización (función process_json)

2.2.2 Deduplicación entre lotes: “la última escritura manda”

2.3 Dataset tabular consolidado: Metrics_limpias.csv como entrada común

2.3.1 Esquema y significado de columnas

2.3.2 Carga rápida y verificación de estructura

2.3.3 Clave primaria compuesta para series temporales

2.4 Fuentes geoespaciales y administrativas: Excel de vertidos y maestro de estaciones

2.4.1 Censo de vertidos (Excel): carga con header=4 y diagnóstico con info()

2.4.2 Maestro de estaciones (JSON): normalización de ID y coordenadas UTM

2.5 Comparación breve de enfoques de ingesta: CSV/Excel/JSON

3. Limpieza y preprocesamiento de datos

3.1 Enfoque general de limpieza y preprocesado en el proyecto

3.2 Inspección estructural inicial: tipos, nulos, duplicados y artefactos

3.2.1 Revisión del DataFrame y diagnóstico de nulos

3.2.2 Eliminación defensiva de artefactos de importación (geoespacial)

3.3 Normalización de esquema y preparación semántica (geoespacial)

3.3.1 Normalización y renombrado de columnas

3.3.2 Eliminación de registros incompletos en campos críticos

3.4 Validación semántica en sensores: reglas físicas y coherencia temporal

3.4.1 Validez física estática (Hard Limits)

3.4.2 Coherencia dinámica: límites de salto por hora (Jump Limits)

3.4.3 Implementación modular y trazable

3.5 Controles adicionales en el pipeline de alto rendimiento (GPU/Parquet)

3.5.1 Negativos y ceros: tratamiento diferenciado

3.5.2 Detección de outliers con Rolling Z-Score: normal vs log-normal

3.6 Imputación determinista y clasificación en variables categóricas (geoespacial)

3.7 Cierre del proceso: criterio de “dataset exportable” y trazabilidad

3.8 Resumen operativo: del dato crudo al dato listo para análisis

4. Análisis estadístico y espacial

4.1 Análisis estadístico y espacial: de la calidad del dato a la topología hidrológica operativa

4.2 Estadística descriptiva y control de calidad del censo

4.2.1 Objetivo: entender distribución, detectar outliers y preparar análisis robustos

4.2.2 Estadísticas numéricas básicas (pandas describe)

4.2.3 Estadísticas categóricas básicas (frecuencias, cardinalidad y sesgos)

4.3 Preparación geoespacial: geometría, CRS y visualización

4.3.1 De tabla a GeoDataFrame (UTM → geometría)

4.3.2 Contextualización con geometría fluvial (WKT → GeoPandas) y reproyección web

4.4 Clustering espacial: detección de hotspots con DBSCAN

4.4.1 Por qué DBSCAN y qué aporta frente a alternativas

4.4.2 Dos parametrizaciones complementarias (5 km vs 2 km)

4.5 Distribución del impacto: Curva de Lorenz, Gini y priorización (TIERs)

4.5.1 Motivación: ¿muchos aportan poco o pocos aportan mucho?

4.5.2 Cálculo de Gini desde Lorenz (integración numérica)

4.6 Topología hidrológica mínima: eje del río, asignación de sensores y relaciones aguas abajo

4.6.1 Objetivo: convertir proximidad en relaciones operativas

4.6.2 Descarga y preparación del eje fluvial (OSM + Overpass)

4.6.3 Asignación de sensores al río por proximidad (con umbral)

4.6.4 Hacia la asignación vertido → sensor aguas abajo

5. Detección de anomalías y valores atípicos

5.1 Detección de anomalías y outliers: un enfoque por capas para calidad de datos

5.2 Capa base: control de outliers para regular sensibilidad antes de modelos avanzados

5.2.1 Umbrales Sigma (Z-Score) como análisis de sensibilidad

5.2.2 De Z-Score global a rolling Z-Score: robustez temporal y estacionalidad corta

5.3 Sensores: detección de anomalías estructurales (flatline) con enfoque multinivel

5.3.1 Enfoque multinivel: de fallos obvios a fallos “contextuales”

5.3.2 Nivel 1 — Repetición exacta (flatline “tonto”)

5.3.3 Nivel 2 — Pérdida de variabilidad (señal “sin vida”)

5.3.4 Nivel 3 — Inconsistencia con el ciclo diario (contexto horario)

5.4. Registros de vertidos: anomalías por “no naturalidad” y outliers contextuales geoespaciales

5.4.1 Auditoría forense de números: Benford y dígito terminal

5.4.2 Outliers multivariantes: magnitud + geografía con Isolation Forest

5.4.3 Anomalías contextuales locales: LOF y explicación por vecinos

5.5 Comparativa breve: cuándo usar cada técnica

6. Transformación y exportación de datos

6.1 Transformación de datos: de la depuración al dataset analítico

6.1.1 Filtrado de calidad: excluir administración pública para mejorar comparabilidad

6.1.2 Preparación del dataset: selección de columnas y tipado numérico/categorico

6.1.3 Preprocesamiento unificado: escalado + One-Hot con ColumnTransformer

6.2 Consolidación: construir artefactos reutilizables y un “DataFrame maestro”

6.2.1 Exportaciones intermedias a CSV como puntos de control

6.2.2 DataFrame maestro para Neo4j: concatenación de “capas” y control de integridad

6.3 Exportación final: del grafo a un reporte tabular listo para análisis

6.3.1 Asignación Vertido–Sensor: estrategia topológica con plan B geométrico

6.3.2 Aplanado de mapas y normalización de campos para evitar colisiones

6.4 Serialización eficiente: Parquet con categorías para grandes datasets de sensores

6.4.1 category + Parquet: Dictionary Encoding automático

6.4.2 Implementación recomendada y particionado por batch

6.4.3 Comparativa breve: CSV vs Parquet en esta suite

7. Análisis y gestión de datos de sensores

7.1 Visión general del flujo: de contexto geoespacial a series temporales confiables

7.2 Contextualización: asignación de vertidos a sensores y relaciones administrativas

7.2.1 Grafo consultable: por qué conectar vertidos, sensores y administración

7.2.2 Rescate por proximidad: asignación del sensor más cercano

7.3 Métricas de presión: carga y “estrés” esperado por sensor

7.3.1 Qué mide el “estrés” y por qué importa para anomalías

7.3.2 “Sensor mártir”: carga exclusiva con filtro de sensores intermedios

7.4 Baseline y calibración: definir “normalidad” antes de limpiar o alertar

7.4.1 Golden Batch: referencia local frente a medias globales

7.4.2 Normal vs. log-normal: tolerancias coherentes con la distribución

7.5 Inspección de anomalías: separar fallos físicos de eventos plausibles

7.5.1 Validación por límites físicos (HARD_LIMITS) y diagnóstico rápido

7.5.2 Contraste con comportamiento plausible: AMONIO dentro de rango

7.6 Segmentación temporal: rupturas por batch y silencios globales

7.6.1 Por qué segmentar antes de calcular volatilidad o imputar

7.6.2 Implementación vectorizada $O(n)$: diff/shift/cumsum

7.7 Deltas y volatilidad “propios”: medir dinámica por sensor dentro de cada segmento

7.7.1 Evitar falsos incrementos por densidad de reportes ajena

7.8 Comparación breve de enfoques: proximidad vs. hidrología, baseline local vs. global

8. Análisis y técnicas avanzadas

8.1 Análisis avanzado geoespacial e integración operativa

8.1.1 Inventario de receptores y priorización de ejes principales

8.2 Construcción de geometrías fluviales reproducibles (OSM/Overpass → UTM)

8.2.1 Persistencia local y control de redundancia

8.2.2 Reproyección a UTM para métricas en metros

8.2.3 Robustez operativa: reintentos y HTTP 429

8.3 Proyección hidráulica: de coordenadas UTM a eje longitudinal (km_rio) y control de calidad (dist_al_rio)

8.3.1 Aplicación homogénea a vertidos y estaciones

8.4 Lógica de asignación por proximidad: “primer sensor aguas abajo” con umbral de ribera

8.4.1 Conceptos y regla de negocio

8.4.2 Implementación (filtrado + ordenación + asignación)

8.5 Capa de atribución: del grafo físico al grafo accionable (titular y municipio)

8.5.1 Modelo de relaciones (expediente como nodo ancla)

8.5.2 Normalización crítica para evitar duplicados

8.6 Cierre: de geometría y texto libre a relaciones reutilizables

9. Calidad y aseguramiento de datos

9.1 Alcance de “Data Quality and Assurance” en series ambientales

9.2 Reglas de dominio: variables con ciclo diario y señales de fallo

9.2.1 Comportamiento esperado en entornos acuáticos

9.3 QC por física estática y dinámica: el “suelo” mínimo de calidad

9.3.1 Hard limits: plausibilidad físico-química

9.3.2 Coherencia temporal: física dinámica (ejemplo explícito)

9.4 Outliers con contexto: Rolling Z-Score (y por qué no basta el z-score global)

9.4.1 Distribuciones típicas: normales vs. cola pesada

9.4.2 Rolling Z-Score: estadística local y segmentación por batch

9.5 Imputación como parte del QA: conservadora primero, avanzada después

9.5.1 Principio de orden: limpiar antes de imputar, e imputar antes de reconstrucciones sensibles

9.5.2 Smart infilling (diferenciado por tipo de variable)

9.5.3 Imputación univariante vs. multivariante: comparación breve

9.5.4 Imputación espacial guiada por topología con Ridge / Bayesian Ridge

10. Conclusiones y elaboración de informes

10.1 Conclusiones del flujo integrado (calidad, continuidad y explotabilidad)

10.2 Gestión de anomalías en sensores: de la detección a la corrección

10.2.1 Detección y neutralización de “flatlines” (señales planas)

10.2.2 Imputación avanzada e iterativa para restaurar continuidad

10.2.3 Control de calidad final del entregable temporal

10.3 Persistencia y escalabilidad: del dataset depurado a artefactos reutilizables

10.3.1 Serialización eficiente (Parquet) como “staging” operativo

10.3.2 Dos niveles de producto: dataset completo vs subconjunto accionable

10.4 Reporting orientado a decisión: “Club TIER 1” y métricas de concentración

10.4.1 Persistencia del subconjunto crítico (“Club TIER 1”)

10.4.2 Métricas y ranking: cuantificar impacto y priorizar acciones

10.4.3 Código mínimo reproducible (extracción y cuota de poder)

11. Referencias bibliográficas

1. Introducción y configuración del entorno

1.1 Contexto del proyecto y motivación

El objetivo del proyecto es desarrollar e implementar un modelo de aprendizaje profundo, basado en una arquitectura DeepONet, una Red de Operadores Profundos (Lu et al., 2021), capaz de resolver el problema inverso de la detección de vertidos en sistemas fluviales. El sistema inferirá un "mapa de calor" de la localización e intensidad de un evento de vertido en tiempo casi real, utilizando como única entrada los datos de series temporales de la red de sensores de calidad del agua existente (32 estaciones).

En el alcance de esta memoria se aborda un problema transversal en datos ambientales y de sensores: **la calidad del dato condiciona por completo la fiabilidad del análisis posterior** (Cabanillas Barbacil, s. f.). En la práctica, los datasets combinan fuentes heterogéneas (registros administrativos, series temporales instrumentales) y suelen contener ruido, inconsistencias y fallos "silenciosos" que no siempre se detectan a simple vista. El objetivo global es **preparar datos robustos** para exploración, visualización, estadística y, especialmente, para **detección de anomalías** sin sesgos inducidos por errores sistemáticos. El trabajo se articula en

tres líneas complementarias que comparten filosofía y herramientas:

- **Análisis geoespacial de autorizaciones de vertido:** ingestión, limpieza, clasificación y visualización cartográfica para descubrir patrones espaciales e irregularidades potenciales en registros de vertidos autorizados.
- **Pipeline de preprocesamiento de alto rendimiento para series de sensores:** transformación y serialización eficiente (Parquet) para manejar grandes volúmenes, con **aceleración GPU** cuando es viable (Alija, 2021).
- **Detección y reparación de fallos “flatline”:** identificación de tramos largos de valores constantes (sensores “atascados”) y corrección mediante validación e **imputación** para recuperar coherencia temporal.

La conexión entre estas líneas es deliberada: los mapas y análisis espaciales dependen de datos limpios; los modelos de anomalías dependen de series temporales coherentes; y el pipeline de alto rendimiento actúa como “capa habilitadora” que convierte datos crudos en un formato consistente y escalable para etapas posteriores (incluida la reparación de flatlines).

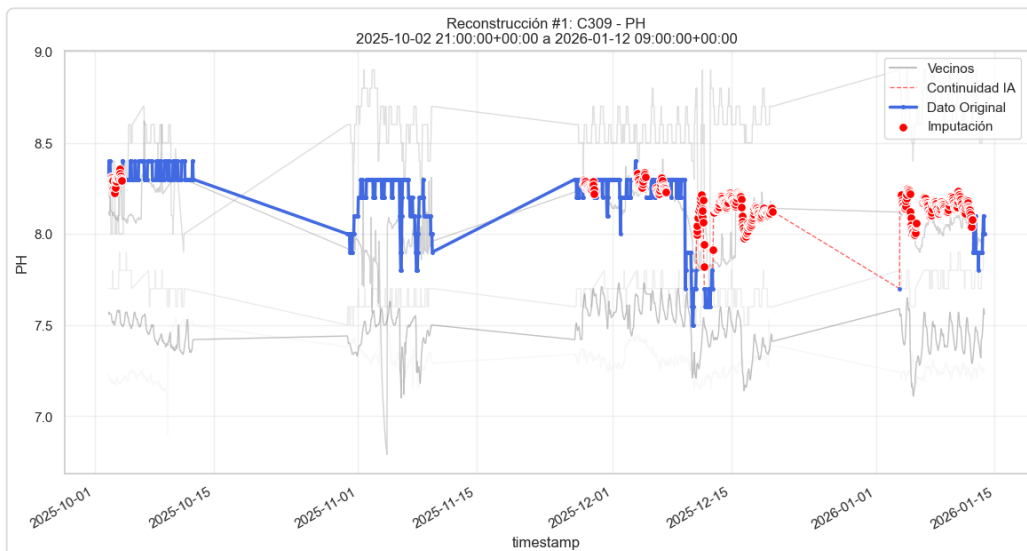
1.2 Fallos “flatline” en series temporales: por qué importan

En sensores ambientales es frecuente un fallo silencioso: el dispositivo no se apaga ni deja de emitir, pero queda **trabado** y reporta el **mismo valor durante periodos prolongados** (minutos u horas). Este patrón, conocido como flatline, degrada la calidad del dato porque:

- **Enmascara eventos reales** (picos, transiciones, cambios de régimen).

- **Distorsiona la detección de anomalías**, generando falsos positivos/negativos (un fallo mecánico puede parecer un “estado estable” o, al revés, un estado estable real puede parecer sospechoso si no se contextualiza).
- **Sesga estadísticas** (medias, varianzas, autocorrelaciones) y validaciones posteriores.

Por ello, una parte clave de la suite se centra en **detectar y corregir** estos tramos constantes para restaurar la validez temporal de la señal. El enfoque no asume que “constante = error” de forma automática: se incorpora una etapa de **validación y control de calidad** para distinguir patrones plausibles (p. ej., caudal cero real) de fallos instrumentales. Esta misma filosofía aparece en otros controles del pipeline, como la “investigación de ceros” (0.0), donde un valor aparentemente sospechoso puede representar un fenómeno real o un error de medición.



[Figura 1.] Reconstrucción de PH en C309

1.3 Enfoque metodológico (visión de alto nivel)

Aunque cada parte profundiza en su dominio, la estrategia general de tratamiento de datos sigue un hilo común: **identificar problemas, validar, corregir y evaluar.**

1.3.1 Detección y reparación de flatlines

La reparación de flatlines se organiza en cuatro pasos:

- **Identificación del fallo:** localizar tramos con repetición anómala de valores y marcarlos como candidatos a corrección.
- **Validación y control de calidad:** filtrar casos donde la constancia podría ser físicamente plausible o contextual (evitando “arreglar” datos correctos).
- **Corrección mediante imputación:** sustituir valores atascados por estimaciones coherentes con el contexto temporal y/o variables relacionadas. Aquí destaca la **imputación iterativa** con `IterativeImputer` de `sklearn`, que puede apoyarse en regresores como Ridge, BayesianRidge o árboles.
- **Evaluación:** cuantificar el impacto de la reparación con métricas como **MAE**, **MSE** y **R²**, verificando que la imputación no introduce artefactos.

Este bloque es especialmente relevante antes de entrenar o ejecutar detectores de anomalías: reduce ruido estructural (fallos mecánicos) y evita que el sistema confunda un flatline con comportamiento ambiental legítimo.

1.3.2 Pipeline de preprocesamiento y serialización (de crudo a listo para análisis)

En paralelo, el pipeline de alto rendimiento está diseñado para convertir datos crudos (grandes, ruidosos, incompletos) en un dataset **consistente y rápido de cargar**, típicamente en **Parquet**, para acelerar exploración, modelado e iteraciones de limpieza. Conceptualmente, el flujo va “de crudo a listo para anomalías”:

- **Ingesta y normalización:** lectura, tipado, estandarización de columnas y formatos.
- **Limpieza y transformaciones masivas:** filtros, joins, agregaciones, creación de features.
- **Serialización eficiente:** escritura a Parquet como producto intermedio (staging) para cargas rápidas y repetibles.
- **Compatibilidad con etapas posteriores:** el output se consume por análisis estadístico, modelos (p. ej., Isolation Forest) y procedimientos avanzados como imputación/flatline cleaner.

1.4 Configuración inicial del entorno de análisis (stack común)

Para sostener estas tareas, la suite utiliza un entorno de trabajo que cubre: manejo de ficheros, análisis tabular, visualización, geoespacial y modelado/anomalías. En el cuaderno geoespacial, por ejemplo, se importan librerías estándar y se fijan ajustes que mejoran la consistencia de la exploración:

- **Sistema y utilidades:** `glob`, `os`, `time` (exploración de directorios, localización de entradas, medición de tiempos).
- **Datos y numérico:** `pandas`, `numpy` (carga, limpieza, agregaciones, transformaciones).
- **Visualización:** `matplotlib`, `seaborn` (EDA y gráficos comparativos).

- **Geoespacial:** `geopandas` , `shapely` (geometrías y operaciones), `folium` y `HeatMap` (mapas interactivos y mapas de calor).
- **Modelado/anomalías y preprocesado:** `IsolationForest` , `LocalOutlierFactor` , `DBSCAN` , y utilidades como `StandardScaler` , `OneHotEncoder` , `Pipeline` , `ColumnTransformer` .
- **Estadística y distancias:** `chisquare` , `cdist` .
- **Conectividad opcional:** `requests` (APIs), `neo4j` (persistencia/consulta en grafos).
- **Explicabilidad:** `shap` (cuando se requiera interpretar modelos).

Ajustes prácticos aplicados al inicio:

- Estilo consistente de gráficos:
`plt.style.use('seaborn-v0_8-darkgrid')` .
- Visualización completa de columnas:
`pd.set_option('display.max_columns', None)` .

Estos componentes se reutilizan según la parte abordada: el análisis geoespacial enfatiza `geopandas/folium` , mientras que el pipeline de series temporales enfatiza serialización y rendimiento.

1.5 Configuración del entorno para procesamiento acelerado por GPU

Cuando el volumen de datos crece, el pipeline prioriza minimizar cuellos de botella de CPU mediante **aceleración por GPU** (RAPIDS). Para que esto sea reproducible y estable, la preparación del entorno se centra en dos ideas: **rutas claras** y **verificación temprana del hardware**.

1.5.1 Librerías del pipeline y rol de cada componente

El stack del pipeline se organiza en tres capas:

- **Manipulación de datos y E/S**
 - `pandas` : operaciones tabulares y validaciones rápidas (especialmente cuando una operación no compensa en GPU).
 - `json` : lectura/escritura de configuraciones y metadatos.
 - `pathlib.Path` : gestión robusta y portable de rutas.
 - `shutil` : operaciones sobre el sistema de ficheros (copias, movimientos, limpieza de staging).
- **Aceleración GPU (RAPIDS + arrays)**
 - `cudf` : DataFrames en GPU (equivalente a `pandas`) para filtros, joins, groupby y transformaciones masivas.
 - `cupy (cp)` : arrays en GPU (equivalente a `numpy`) para cálculo numérico y features vectorizadas.
 - `numpy` : compatibilidad y fallback cuando no hay soporte GPU o no compensa.
- **Gestión de recursos**
 - `rmm` : RAPIDS Memory Manager para controlar memoria GPU, reducir fragmentación y estabilizar ejecuciones largas.
 - `gc` : recolección de basura en CPU para evitar picos de memoria en pipelines iterativos.

Para detección de anomalías se usa `sklearn.ensemble.IsolationForest` como baseline en CPU, especialmente útil cuando una alternativa GPU (p. ej., cuML) no está disponible o no es estable en el entorno. **Comparación breve (CPU vs GPU):** `pandas/numpy` ofrecen máxima compatibilidad y simplicidad; `cudf/cupy` aportan rendimiento en transformaciones masivas, pero requieren verificación de GPU, control de memoria y mayor disciplina en rutas/artefactos para evitar ejecuciones costosas fallidas. > Nota técnica: si aparece una importación de `dropna` desde módulos internos de testing de pandas, en documentación/producción debe evitarse. Para controlar nulos en

`groupby`, es preferible usar APIs públicas o filtrar nulos explícitamente.

1.5.2 Rutas del proyecto y limpieza segura de artefactos

Antes de procesar, se definen rutas explícitas para separar **entrada cruda** y **salida procesada**:

- `BASE_DIR` : ubicación del dataset **raw** (punto de entrada).
- `OUTPUT_DIR` : destino del dataset **procesado** en Parquet (por ejemplo, `measurements.parquet`), que actúa como producto intermedio para cargas rápidas.

Un paso crítico es **limpiar el directorio de salida** si ya existe, para evitar mezclar fragmentos antiguos con nuevos (muy importante si cambia el esquema, tipos de columnas o el particionado tipo “Hive”). Código esencial:

```
from pathlib import Path
import shutil

BASE_DIR = Path("~/Projects/proyecto-computacion-I/
data/datasets/raw").expanduser()
OUTPUT_DIR = Path("~/Projects/proyecto-computacion-I/
data/datasets/processed/
measurements.parquet").expanduser()

# Limpieza previa de archivos: útil si cambian nombres
de atributos o el particionado (Hive style)
if OUTPUT_DIR.exists():
    print(f"Limpiando directorio antiguo:
{OUTPUT_DIR}")
    shutil.rmtree(OUTPUT_DIR)
```

Salida esperada:

```
Limpiando directorio antiguo: /home/duo/Projects/  
proyecto-computacion-I/data/datasets/processed/  
measurements.parquet
```

Este tipo de confirmación suele verse como una celda de salida.

Nota operativa: en entornos colaborativos o productivos, conviene añadir salvaguardas (por ejemplo, validar que `OUTPUT_DIR` pertenece a una lista de directorios permitidos) para evitar borrados accidentales.

1.5.3 Verificación de GPU antes de cargar librerías aceleradas

Para reducir fallos tardíos (cuando ya se han iniciado transformaciones costosas), se recomienda una verificación temprana del hardware NVIDIA mediante NVML (vía `pynvml`). Esto permite:

- confirmar que el sistema detecta GPUs,
- conocer cuántas hay disponibles,
- registrar el modelo (útil para trazabilidad y debugging de rendimiento).

Esta comprobación se ejecuta **antes** de depender de cuDF/RAPIDS, de modo que el pipeline pueda degradar a CPU o abortar con un mensaje claro si no hay GPU disponible. Con esta base —objetivos claros, control de calidad (incluido flatline), y un entorno reproducible con rutas y verificación de recursos— el resto de la suite puede centrarse en lo esencial: transformar datos ambientales complejos en insumos confiables para mapas, estadística y detección de anomalías.

2. Carga de datos y exploración inicial

2.1 Carga de datos y exploración inicial

La fase de Data Loading and Initial Exploration establece una base común para todo el proyecto: antes de limpiar, serializar (Parquet/GPU), mapear o detectar anomalías (p. ej., flatlines), se valida que cada fuente entra con un **esquema estable**, **tipos correctos** y **campos clave completos**. En este flujo conviven tres familias de datos: (1) series temporales de sensores (JSON → tabla homogénea), (2) un dataset tabular ya consolidado para análisis posteriores (`Metricas_limpias.csv`), y (3) fuentes geoespaciales/administrativas (censo de vertidos en Excel y maestro de estaciones en JSON) que permiten contextualizar lecturas en el espacio y cruzarlas con autorizaciones.

2.1.1 Objetivo transversal: esquema, tipado y completitud

Independientemente del formato (CSV/Excel/JSON), la exploración inicial persigue tres verificaciones mínimas:

- **Esquema:** confirmar columnas esperadas y nombres coherentes (evita roturas en pipelines que asumen campos fijos).

- **Tipado:** convertir explícitamente fechas, valores numéricos y coordenadas (reduce errores y mejora rendimiento, especialmente si se serializa a Parquet o se usa cuDF).
- **Compleitud y claves:** detectar nulos relevantes, duplicados o IDs inconsistentes que puedan provocar joins multiplicativos o análisis sesgados.

A partir de estas comprobaciones se decide si hay que normalizar nombres, filtrar registros inválidos o aplicar reglas de deduplicación (“última escritura manda”).

2.2 Series temporales de sensores: de JSON a tabla homogénea

Cuando las lecturas llegan como múltiples ficheros JSON (organizados por directorios `batch*`), el objetivo inicial no es “analizar” todavía, sino **convertir** cada JSON a un DataFrame aplanado y concatenable: una fila por medición, con contexto mínimo (sensor, métrica, unidad, lote) y tipos consistentes.

2.2.1 Parseo y normalización (función `process_json`)

La pieza central es una función que encapsula extracción, aplanado y tipado. Esto es crítico porque el siguiente paso suele ser serialización eficiente (Parquet) y/o procesamiento acelerado (cuDF), donde los tipos inconsistentes penalizan rendimiento o generan errores. Aspectos clave del parseo:

- **Metadatos globales:** desde `response` se extraen `nombre` (métrica) y `unidad`.
- **Observaciones:** `response.valores` se transforma en DataFrame.
- **Tipado explícito:**
- `tiempo` → `timestamp` como `datetime`.

- `valor` → `value` como numérico, forzando `NaN` si no es convertible (`errors='coerce'`).
- **Trazabilidad:** se añaden `metric` , `unit` , `batch` .
- **Identificación de sensor:** si no viene en el contenido, se infiere del nombre del archivo (p. ej., `C302_AMONIO.json` → `C302`).
- **Esquema mínimo estable:** se seleccionan columnas esenciales para mantener uniformidad y reducir tamaño.

Código representativo (tipado + columnas mínimas + sensor desde filename):

```

def process_json(filepath, batch_name):
    """Lee un JSON y devuelve un DataFrame aplanado."""
    with open(filepath, 'r') as f:
        data = json.load(f)

    # Extraer metadatos globales del archivo
    resp = data.get('response', {})
    metric_name = resp.get('nombre')
    unit = resp.get('unidad')

    # Extraer la lista de valores
    valores = resp.get('valores', [])

    if not valores:
        return pd.DataFrame()

    # Crear DataFrame
    df = pd.DataFrame(valores)

    # Limpieza y Tipado (CRUCIAL para Parquet/cuDF)
    if 'tiempo' in df.columns:
        df['timestamp'] = pd.to_datetime(df['tiempo'],
format='%d/%m/%Y %H:%M')

    if 'valor' in df.columns:
        df['value'] = pd.to_numeric(df['valor'],
errors='coerce')

    # Añadir columnas de contexto
    df['metric'] = metric_name
    df['unit'] = unit
    df['batch'] = batch_name

```

En la práctica, tras procesar todos los JSON de todos los lotes, se concatenan en una única tabla para habilitar operaciones lineales ($O(N)$) de control de calidad, deduplicación y serialización.

2.2.2 Deduplicación entre lotes: “la última escritura manda”

Al integrar múltiples lotes (`batch*`) es común que existan solapes temporales o reenvíos. Por eso se adopta una regla determinista: **si hay duplicados de la misma medición, prevalece el registro más reciente** (última escritura). Esta decisión se apoya en una clave temporal y de entidad (sensor + métrica + timestamp) y evita inconsistencias cuando se re-procesan lotes o se corrigen datos aguas arriba.

2.3 Dataset tabular consolidado: `Metricas_limpias.csv` como entrada común

Una vez estandarizadas las lecturas (o cuando ya se dispone de una versión depurada), el flujo converge en un dataset base que actúa como “contrato” para etapas posteriores de QA, serialización y detección/corrección de anomalías.

2.3.1 Esquema y significado de columnas

El fichero `Metricas_limpias.csv` consolida lecturas temporales en un formato tabular homogéneo. Su esquema incluye tanto campos “crudos” como señales auxiliares para análisis:

- `timestamp` : instante de medición (eje temporal para ordenación, resampling y detección de patrones).
- `value` : valor observado original.
- `sensor_id` : identificador del sensor/estación.

- `metric` : variable medida.
- `unit` : unidad de la métrica.
- `batch` : etiqueta de lote/periodo/campaña (útil para comparar condiciones “normales” vs. desviaciones).
- `trans_value` : valor transformado (p. ej., estabilización de varianza o preparación para detección de outliers).
- `z_score_abs` : magnitud del z-score absoluto (señal auxiliar para priorizar revisiones/anomalías).

Además, el catálogo de métricas (`metric`) cubre parámetros ambientales diversos (p. ej., PH, AMONIO, TEMPERATURA, OXIGENO DISUELTO, TURBIDEZ, etc.), lo que habilita tanto análisis multivariable como detección de fallos específicos por sensor o métrica.

2.3.2 Carga rápida y verificación de estructura

La exploración inicial aquí es deliberadamente simple: cargar, comprobar columnas y validar el catálogo de métricas. Estas comprobaciones tempranas evitan ejecutar etapas posteriores sobre datos con cambios de esquema o dominios inesperados.

```
import pandas as pd

df = pd.read_csv("/home/duo/Projects/proyecto-
computacion-I/data/datasets/Metricas_limpias.csv")

df.columns
```

Salida esperada (estructura detectada):

```
Index(['timestamp', 'value', 'sensor_id', 'metric',  
      'unit', 'batch',  
      'trans_value', 'z_score_abs'],  
      dtype='object')
```

Validación del dominio (métricas presentes):

```
df.metric.unique()
```

Salida típica:

```
array(['PH', 'AMONIO', 'CLOROFILA', 'CARBONO ORGANICO',  
      'OXIGENO DISUELTO', 'CONDUCTIVIDAD', 'TURBIDEZ',  
      'TEMPERATURA',  
      'FICOCIANINAS', 'NIVEL', 'FOSFATOS',  
      'NITRATOS'], dtype=object)
```

Cabe destacar que no todas las estaciones recogen todas las métricas, se ha agrupado por tipos de sensores y se han encontrado 13 configuraciones diferentes

```
[('AMONIO', 'CONDUCTIVIDAD', 'NITRATOS', 'OXIGENO
DISUELTO', 'PH', 'TEMPERATURA', 'TURBIDEZ')
 ('CLOROFILA', 'CONDUCTIVIDAD', 'FICOCIANINAS',
 'OXIGENO DISUELTO', 'PH', 'TEMPERATURA', 'TURBIDEZ')
 ('AMONIO', 'CARBONO ORGANICO', 'CONDUCTIVIDAD',
 'FOSFATOS', 'NIVEL', 'OXIGENO DISUELTO', 'PH',
 'TEMPERATURA', 'TURBIDEZ')
 ('AMONIO', 'CARBONO ORGANICO', 'CONDUCTIVIDAD',
 'NIVEL', 'OXIGENO DISUELTO', 'PH', 'TEMPERATURA',
 'TURBIDEZ')
 ('AMONIO', 'CONDUCTIVIDAD', 'NITRATOS', 'NIVEL',
 'OXIGENO DISUELTO', 'PH', 'TEMPERATURA', 'TURBIDEZ')
 ('AMONIO', 'CARBONO ORGANICO', 'CONDUCTIVIDAD',
 'OXIGENO DISUELTO', 'PH', 'TEMPERATURA', 'TURBIDEZ')
 ('AMONIO', 'CARBONO ORGANICO', 'CONDUCTIVIDAD',
 'FOSFATOS', 'NITRATOS', 'NIVEL', 'OXIGENO DISUELTO',
 'PH', 'TEMPERATURA', 'TURBIDEZ')
 ('CONDUCTIVIDAD', 'NIVEL', 'OXIGENO DISUELTO', 'PH',
 'TEMPERATURA', 'TURBIDEZ')
 ('TEMPERATURA',)
 ('CONDUCTIVIDAD', 'OXIGENO DISUELTO', 'PH',
 'TEMPERATURA')
 ('CONDUCTIVIDAD', 'OXIGENO DISUELTO', 'PH',
 'TEMPERATURA', 'TURBIDEZ')
 ('CARBONO ORGANICO', 'CONDUCTIVIDAD', 'NIVEL',
 'OXIGENO DISUELTO', 'PH', 'TEMPERATURA', 'TURBIDEZ')
 ('AMONIO', 'CARBONO ORGANICO', 'CONDUCTIVIDAD',
 'FOSFATOS', 'NITRATOS', 'OXIGENO DISUELTO', 'PH',
 'TEMPERATURA', 'TURBIDEZ')]
```

2.3.3 Clave primaria compuesta para series temporales

Para tratar el dataset como una colección de series temporales coherentes, se adopta una **clave primaria compuesta**:

- (sensor_id, metric, timestamp)

Esta clave guía decisiones posteriores (deduplicación determinista, ordenación temporal, imputación y detección de patrones como flatlines). También permite definir filtros desde el inicio por `sensor_id`, `metric` y `batch`, lo que simplifica exploraciones focalizadas y comparaciones entre campañas.

2.4 Fuentes geoespaciales y administrativas: Excel de vertidos y maestro de estaciones

La exploración inicial no se limita a series temporales: el proyecto también analiza autorizaciones de vertido y necesita enlazar lecturas con ubicaciones. Por ello se cargan y validan dos piezas: el **censo de vertidos** (Excel) y el **maestro de estaciones** (JSON con coordenadas). Estas autorizaciones se enmarcan en la normativa estatal sobre aguas y vertidos (España, 2001)

2.4.1 Censo de vertidos (Excel): carga con header=4 y diagnóstico con info()

El censo se ingiere desde un Excel (`Censo vertidos.xlsx`) y se inspecciona para entender estructura, calidad y campos disponibles antes de limpiar nulos, normalizar nombres o preparar coordenadas UTM para proyección y mapeo. Un detalle técnico importante es que los encabezados reales están en la fila 5 del Excel, por lo que se usa `header=4` para omitir filas iniciales

(metadatos/títulos/notas) y evitar contaminar el esquema. La exploración inicial típica combina:

- `df.head()` para confirmar alineación de columnas/valores (y detectar desplazamientos por celdas combinadas).
- `df.info()` para tamaño, tipos y no nulos.

Resultados observados en la carga:

- **1837 entradas y 13 columnas**
- Tipos: 9 **object** y 4 **float64**
- Memoria aproximada: **186.7 KB**
- Presencia de una columna vacía tipo índice (**Unnamed: 0** con 0 no nulos), candidata a eliminar en limpieza.

Fragmento relevante de `df.info()` :

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1837 entries, 0 to 1836
Data columns (total 13 columns):
#   Column                                Non-Null Count  Dtype
---  -
0   Unnamed: 0                            0 non-null      float64
1   NOMBRE DEL VERTIDO                    1836 non-null   object
...
5   UTM X
Huso 30                                1835 non-null   float64
6   UTM Y
Huso 30                                1835 non-null   float64
...
10  VOLUMEN (m3/año)                      1835 non-null   float64
...
dtypes: float64(4), object(9)
memory usage: 186.7+ KB
```

Esta salida se documenta visualmente para justificar decisiones de limpieza posteriores (p. ej., `df.info()` `Censo vertidos`).

2.4.2 Maestro de estaciones (JSON): normalización de ID y coordenadas UTM

Para conectar lecturas temporales con ubicación (mapas, análisis espacial y cruces con vertidos), se utiliza un **maestro de estaciones** (`df_stations` / `stations`) con identificadores y coordenadas. La exploración inicial se centra en asegurar que:

- el identificador de estación es consistente con otras tablas (campo de unión),
- `utm_x` y `utm_y` son numéricos,
- no se arrastran estaciones sin coordenadas válidas.

Patrón recomendado (normalización + tipado robusto + filtrado):

```
import pandas as pd

stations = pd.read_json("/tmp/pycharm_project_992/data/
datasets/raw/saica_stations_maestro_coords.json")

# Normalización de identificador para unificar con
otras tablas
stations = stations.rename(columns={'codigo':
'id_estacion'})

# Tipado robusto de coordenadas (valores no parseables
-> NaN)
stations['utm_x'] = pd.to_numeric(stations['utm_x'],
errors='coerce')
stations['utm_y'] = pd.to_numeric(stations['utm_y'],
errors='coerce')

# Filtrado de registros sin coordenadas válidas
stations = stations.dropna(subset=['utm_x', 'utm_y'])
```

Tras esta normalización, la exploración rápida recomendada incluye `head()`, conteo de nulos, detección de duplicados en `id_estacion` y verificación de rangos UTM (para identificar ceros o magnitudes inverosímiles). Estas comprobaciones previenen errores posteriores en proyecciones, joins y visualizaciones cartográficas.

2.5 Comparación breve de enfoques de ingesta: CSV/Excel/JSON

Aunque las fuentes difieren, el criterio de calidad es el mismo:

- CSV (`Metricas_limpias.csv`): ideal como “contrato” estable; la exploración se centra en confirmar columnas y dominio

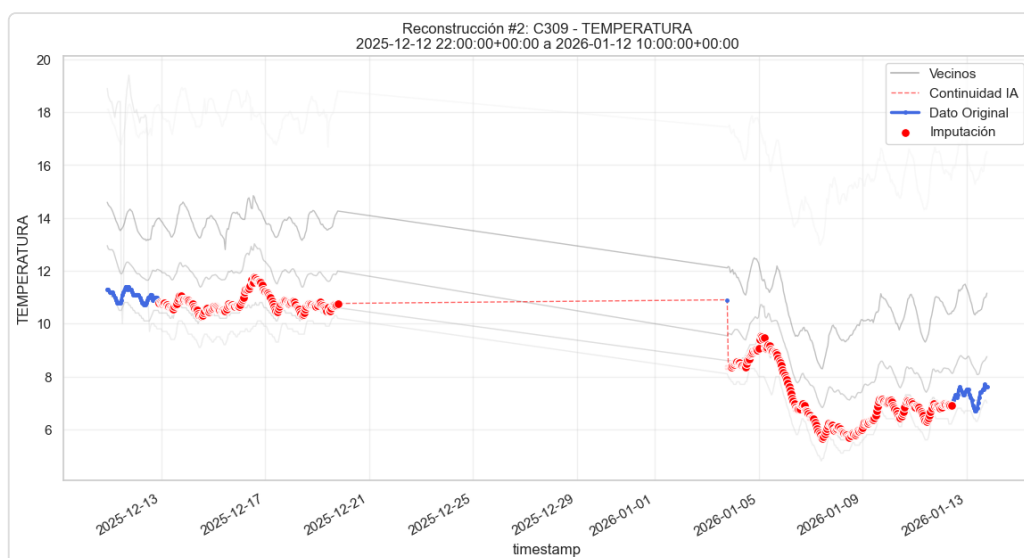
(`metric`), porque el dataset ya está tabular y listo para etapas de QA/anomalías.

- **JSON (lecturas por lotes):** requiere mayor normalización (aplanado, tipado, enriquecimiento con contexto y deduplicación), pero permite construir el dataset base de forma reproducible y escalable.
- **Excel (censo de vertidos):** suele exigir atención a encabezados no estándar (`header=4`) y a columnas “basura” (`Unnamed`), por lo que `head()` / `info()` son esenciales para decidir limpieza.
- **JSON maestro de estaciones:** se orienta a asegurar join keys y coordenadas numéricas; el filtrado temprano de estaciones sin UTM evita fallos en mapas y análisis espacial.

Con estas bases validadas, el pipeline puede avanzar con confianza hacia limpieza avanzada, serialización eficiente (Parquet/GPU) y detección/corrección de anomalías temporales, manteniendo trazabilidad por `batch` , `sensor_id` y `metric` .

3. Limpieza y preprocesamiento de datos

3.1 Enfoque general de limpieza y preprocesado en el proyecto



[Figura 2.]¹

La suite aborda problemas de calidad de datos en dos dominios complementarios: (1) registros administrativos/geoespaciales de

1. Gráfico lineal de temperatura a lo largo del tiempo (12-12-2025 al 12-01-2026). El azul indica los datos originales, los puntos rojos y la línea discontinua roja muestran imputaciones y continuidad de la IA, mientras que las líneas grises representan reconstrucciones vecinas; la temperatura cae de unos 11°C a finales de diciembre a unos 6–7°C a principios de enero, con valores imputados rellenos los huecos.

autorizaciones de vertido y (2) series temporales de sensores ambientales orientadas a detección de anomalías y reparación de fallos (p. ej., flatlines). En ambos casos, el objetivo de Data Cleaning and Preprocessing es el mismo: **convertir datos “crudos” en un dataset consistente, trazable y listo para análisis**, minimizando sesgos y evitando que errores de captura/parseo contaminen estadísticas, mapas o modelos. La diferencia principal es el tipo de validación dominante:

- En **geoespacial**, la limpieza se centra en **calidad estructural y semántica del registro** (columnas consistentes, eliminación de artefactos de importación, completitud de campos críticos y clasificación categórica).
- En **sensores**, la limpieza incorpora además **validación física y coherencia temporal**, y se apoya en controles masivos (incluyendo GPU/RAPIDS) para escalar a cientos de miles de filas.

Un principio transversal del proyecto es **no borrar evidencia durante la validación**: cuando un valor es sospechoso o inválido, se **marca** (típicamente como **NaN**) para que las fases posteriores (imputación, reparación de series, detección de anomalías) trabajen sobre un dataset coherente sin perder auditabilidad. La eliminación de filas se reserva para el **momento de exportación**, cuando se necesita un dataset “consumible” sin mediciones vacías.

3.2 Inspección estructural inicial: tipos, nulos, duplicados y artefactos

Antes de aplicar reglas físicas, imputaciones o detección de outliers, se realiza una inspección rápida para identificar puntos débiles: columnas inconsistentes, índices serializados, tipos incorrectos, nulos y duplicados. Este paso actúa como “control de calidad” temprano y reduce errores aguas abajo.

3.2.1 Revisión del DataFrame y diagnóstico de nulos

En el censo geoespacial, `df.info()` se utiliza para entender tamaño, tipos y completitud. Tras la limpieza estructural, el dataset queda en el orden de **1835 registros** y alrededor de **12–13 columnas**, con mezcla de variables numéricas (`float64`) y categóricas (`object`). Un hallazgo relevante es que, una vez estabilizadas las columnas principales, la **única columna con nulos** pasa a ser `tipo_titular` (una variable derivada de clasificación), lo que permite focalizar el tratamiento de faltantes en un único punto. Cuando se documenta esta inspección, conviene acompañarla de la salida de la celda correspondiente (por ejemplo,

) para dejar evidencia del estado del dataset tras la limpieza. En el pipeline de sensores orientado a Parquet/GPU, la inspección estructural se formaliza con dos controles clave:

- **Duplicados:** se define una clave primaria compuesta (`sensor_id`, `metric`, `timestamp`) y se valida la unicidad con `drop_duplicates(...)`. En la carga analizada ($\approx 278,037$ filas), no se detectan duplicados, lo que confirma que la granularidad temporal por sensor/métrica es consistente.
- **Nulos:** se cuantifican con `isna().sum()` para detectar huecos introducidos por parseo de tipos o lecturas incompletas. En esa carga concreta, el resultado es **0 nulos**, lo cual es valioso porque separa “problemas reales del fenómeno” de “problemas accidentales de ingestión”.

Esta inspección inicial es deliberadamente simple: su función es asegurar integridad estructural antes de pasar a validaciones semánticas (física del fenómeno, coherencia temporal, distribución estadística).

3.2.2 Eliminación defensiva de artefactos de importación (geoespacial)

En fuentes CSV/Excel es común arrastrar el índice como columna (`Unnamed: 0`). Para evitar duplicidad de identificadores y ruido en análisis, se elimina de forma defensiva:

```
if 'Unnamed: 0' in df.columns:  
    df = df.drop(columns=['Unnamed: 0'])
```

Este patrón es pequeño pero importante: evita que operaciones posteriores (joins, agrupaciones, exportaciones) se basen accidentalmente en un “pseudo-id” sin significado.

3.3 Normalización de esquema y preparación semántica (geoespacial)

Una vez controlados artefactos y estructura, el censo geoespacial prioriza que el dataset sea **fácil de referenciar** y **semánticamente consistente** para visualización y análisis territorial.

3.3.1 Normalización y renombrado de columnas

Se normalizan nombres de columnas para eliminar espacios extra, saltos de línea y caracteres problemáticos, y después se renombran campos clave a nombres cortos y estables (por ejemplo, `VOLUMEN (m3/año)` → `VOLUMEN_m3_ano`, `UTM X Huso 30` → `UTM_X`). Esta estandarización reduce errores en código y mejora la legibilidad en operaciones típicas: agrupaciones por municipio/provincia, filtros, generación de mapas y cálculos de métricas.

3.3.2 Eliminación de registros incompletos en campos críticos

A diferencia del enfoque de sensores (donde se prefiere marcar como **NaN**), en el censo geoespacial se eliminan filas con nulos en campos imprescindibles para el análisis:

- **TITULAR** (necesario para análisis por responsable y clasificación)
- **MUNICIPIO** (necesario para análisis territorial)
- **VOLUMEN_m3_ano** (variable cuantitativa central)

Aquí la eliminación es razonable porque el registro administrativo sin esos campos pierde gran parte de su utilidad analítica y puede distorsionar agregaciones territoriales.

3.4 Validación semántica en sensores: reglas físicas y coherencia temporal

En series temporales de sensores, la limpieza debe capturar errores que no son solo “faltantes”, sino **mediciones físicamente imposibles** o **dinámicamente incoherentes**. Por eso se aplican reglas deterministas antes de imputar o detectar anomalías.

3.4.1 Validez física estática (Hard Limits)

Se define un diccionario de rangos aceptables por métrica. Cualquier valor fuera de rango se considera físicamente imposible o altamente improbable y se invalida marcándolo como **NaN** (sin borrar filas). Ejemplos de rangos configurados:

- **PH:** 4.0 a 10.5
- **CONDUCTIVIDAD:** 20.0 a 5000.0
- **CLOROFILA:** 0.001 a 500.0
- **OXIGENO DISUELTO:** 0.001 a 20.0

- **TEMPERATURA:** 0.001 a 38.0

Este control evita que lecturas típicas de fallo (por ejemplo, **pH = 14.0**, asociado a saturación o error de sensor) contaminen estadísticas, umbrales o modelos. La decisión de invalidar (en lugar de eliminar) preserva trazabilidad: el “hueco” queda explícito y puede imputarse o auditarse después.

3.4.2 Coherencia dinámica: límites de salto por hora (Jump Limits)

Incluso dentro de rango, una métrica puede ser sospechosa si cambia demasiado rápido. Para ello se aplican límites de variación máxima por hora en métricas sensibles:

- **TEMPERATURA:** máximo 4.0 °C/h
- **PH:** máximo 2.0 unidades/h
- **OXIGENO DISUELTO:** máximo 5.0 ppm/h
- **CLOROFILA:** máximo 20 (unidades/h según escala del sensor)

La lógica es detectar reinicios, burbujas/interferencias, saturación o errores de transmisión que se manifiestan como saltos bruscos. Para calcular correctamente estos saltos, el dataset debe estar **ordenado por** `sensor_id`, `metric`, `timestamp`, y el diferencial debe computarse **agrupando por sensor** para no comparar series distintas. La técnica central es `groupby().diff()` aplicada sobre el valor temporalmente ordenado.

3.4.3 Implementación modular y trazable

La limpieza se encapsula en una función tipo `clean_physics_and_coherence(df)` que combina ambos controles (hard limits + jump limits) y reporta el impacto: cuántos valores se invalidan y cuánto crece el número total de `NaN`. Aunque se dispone del esqueleto de la función, el punto crítico del diseño es la modularidad: permite ejecutar la misma validación de forma

reproducibles antes de imputación avanzada o reparación de flatlines.

3.5 Controles adicionales en el pipeline de alto rendimiento (GPU/Parquet)

Cuando el objetivo es preparar grandes volúmenes para serialización (Parquet) y análisis posterior, se añaden comprobaciones rápidas que detectan errores evidentes y cuantifican patrones sospechosos.

3.5.1 Negativos y ceros: tratamiento diferenciado

- **Valores negativos:** para métricas como pH, turbidez o nutrientes, los negativos no son físicamente plausibles. En la carga analizada se detectan **28 mediciones negativas**, todas en **FICOCIANINAS**. En coherencia con el enfoque global, deben **marcarse como inválidas** (p. ej., **NaN**) para imputación posterior.
- **Valores cero:** se detectan **41,492 mediciones con 0.0**. A diferencia de los negativos, el cero puede ser válido (concentración muy baja) o indicar fallo (sensor bloqueado, desconexión, cuantización). Por ello, el pipeline no los invalida automáticamente: los **identifica y cuantifica** para cruzarlos con otras señales (por ejemplo, detección de flatlines o periodos de mantenimiento).

Este contraste (negativo → invalidez casi segura; cero → depende del contexto) evita reglas demasiado agresivas que podrían eliminar eventos reales.

3.5.2 Detección de outliers con Rolling Z-Score: normal vs log-normal

Para detectar anomalías compatibles con eventos raros (p. ej., vertidos), se aplica un enfoque estadístico por sensor y métrica basado en **Rolling Z-Score**, ajustando el supuesto de distribución:

- Aproximadamente normal: PH , TEMPERATURA , OXIGENO DISUELTO , NIVEL
- Cola pesada / log-normal: AMONIO , TURBIDEZ , NITRATOS , etc.

La idea es que una métrica con cola pesada produce “outliers” aparentes incluso en condiciones normales; por eso conviene transformar o modelar bajo supuestos log-normales para reducir falsos positivos. Este enfoque convive con las reglas deterministas: primero se asegura validez física y coherencia, y después se buscan desviaciones estadísticas sobre una señal ya depurada.

3.6 Imputación determinista y clasificación en variables categóricas (geoespacial)

En el censo, el problema de calidad dominante no es físico-temporal sino de **clasificación incompleta**. Tras la inspección, `tipo_titular` aparece como el único campo con nulos (por ejemplo, 1490 no nulos de 1835), lo que sugiere que el fallo no es aleatorio sino atribuible a reglas de clasificación insuficientes o a variaciones en el texto. La estrategia aplicada es **imputación determinista y trazable**: reconstruir `tipo_titular` a partir de `TITULAR` mediante reglas por palabras clave. Antes de imputar, se agrupan los nulos por `TITULAR` para detectar patrones repetidos (p. ej., entidades frecuentes sin clasificar como “Canal de Isabel II”), lo que ayuda a mejorar reglas y a justificar la imputación como corrección semántica, no como “relleno estadístico”. Este enfoque

contrasta con la imputación en sensores: allí se prioriza reparación temporal (interpolación, métodos avanzados) tras invalidar valores; aquí se prioriza consistencia categórica para permitir análisis por tipo de responsable.

3.7 Cierre del proceso: criterio de “dataset exportable” y trazabilidad

Una vez aplicadas validaciones, marcados de invalidez y (cuando corresponde) imputaciones o reparaciones, el proyecto distingue entre:

- **Dataset de trabajo (`df_final`)**: conserva el histórico del proceso, incluyendo `NaN` como marca explícita de invalidez o de dato no recuperable.
- **Dataset exportable (`df_export`)**: se entrega sin filas donde falte la señal esencial.

En series temporales, el criterio de exportación se centra en `value`: **no se exportan registros sin medición válida**. Se aplica una regla mínima y transparente:

```
df_export = df_final.dropna(subset=['value'])
```

Esto mantiene coherencia con el principio global: durante la validación no se borran filas (se marcan valores), pero al exportar se produce un dataset listo para consumo por terceros o por pipelines aguas abajo (modelado, detección de anomalías, serialización eficiente). En un notebook, la exportación a CSV queda comentada para evitar sobrescrituras accidentales y permitir controlar rutas/permisos:

```
# df_export.to_csv("/home/duo/Projects/proyecto-  
computacion-I/data/datasets/Métricas_inputadas_.csv")
```

Como práctica recomendada, se documenta el impacto del filtrado (filas antes/después) y se revisan nulos residuales en columnas críticas, especialmente si el destino es un pipeline de alto rendimiento (Parquet/GPU) donde los errores se amplifican por volumen.

3.8 Resumen operativo: del dato crudo al dato listo para análisis

El flujo unificado que emerge de cada parte tratada es:

- Geoespacial: completitud en campos críticos + clasificación determinista.
- Sensores: hard limits + jump limits + controles de negativos/ceros.

Este diseño equilibra dos necesidades normalmente en tensión: **limpiar lo suficiente para que el análisis sea fiable**, sin perder la evidencia necesaria para auditar fallos del sensor, problemas de ingestión o inconsistencias administrativas.

4. Análisis estadístico y espacial

4.1 Análisis estadístico y espacial: de la calidad del dato a la topología hidrológica operativa

El análisis estadístico y espacial constituye el “puente” entre un censo administrativo de autorizaciones de vertido (puntos con atributos) y un sistema de vigilancia ambiental basado en sensores y relaciones aguas arriba/abajo. El objetivo no es solo describir el dataset, sino **validarlo, detectar irregularidades, identificar patrones de concentración** y, finalmente, **convertir geometría y proximidad en relaciones operativas** (qué sensores podrían observar qué vertidos y con qué prioridad). En el flujo global, esta sección se organiza en cuatro capas que se encadenan: (1) estadística descriptiva y control de calidad, (2) preparación geoespacial y clustering de densidad, (3) análisis de concentración del impacto (Lorenz/Gini) para priorización, y (4) digitalización del eje fluvial y asignación sensores–río–vertidos para construir una topología mínima reutilizable en análisis posteriores (incluido soporte a detección de anomalías en series temporales).

4.2 Estadística descriptiva y control de calidad del censo

4.2.1 Objetivo: entender distribución, detectar outliers y preparar análisis robustos

Antes de aplicar técnicas espaciales (clustering, asignación a red fluvial) o métricas de desigualdad (Lorenz/Gini), es crítico evaluar **completitud, escalas, dispersión y consistencia categórica** (Pérez Juste, 2012). Esta fase combina:

- **Estadísticas numéricas** para variables continuas (p. ej., `VOLUMEN_m3_ano`).
- **Estadísticas categóricas** para variables discretas (p. ej., provincia, naturaleza del vertido, medio receptor).

El objetivo práctico es doble: (a) anticipar sesgos que distorsionen agregaciones (por ejemplo, medias dominadas por pocos valores extremos) y (b) detectar campos que requieran normalización (alta cardinalidad, errores tipográficos, codificaciones inconsistentes).

4.2.2 Estadísticas numéricas básicas (pandas describe)

Para columnas numéricas, el resumen estándar de `pandas` permite detectar rápidamente extremos y asimetrías:

```
display(df.describe())
```

Interpretación clave en este dataset: el volumen de vertido muestra **dispersión extrema**, con **media ~914.000 m³** y **desviación estándar ~1.8×10⁷**. Esta señal suele indicar una mezcla de muchos

vertidos pequeños con unos pocos muy grandes (p. ej., grandes EDARs o industria), y tiene implicaciones directas:

- la **media** puede dejar de ser representativa;
- conviene usar medidas **robustas** (mediana, IQR) y/o transformaciones (p. ej., **log**);
- los máximos deben revisarse por posibles **errores de unidades** o registros anómalos.

4.2.3 Estadísticas categóricas básicas (frecuencias, cardinalidad y sesgos)

Para variables tipo texto/categoría, el resumen ayuda a ver dominancias y valores raros:

```
display(df.describe(include='object'))
```

Hallazgos relevantes para orientar el análisis espacial posterior:

- **Provincia:** Madrid concentra el mayor número de vertidos registrados (585), lo que sugiere un foco espacial/administrativo claro para inspección o muestreo.
- **Naturaleza del vertido:** predominan **URBANO O ASIMILABLE (1.711 de 1.835)**. Esto condiciona la lectura de patrones: la señal industrial puede estar infra-representada en conteo, pero no necesariamente en volumen.
- **Medio receptor:** el más común es **TERRENO (912 casos)**, lo que anticipa una dificultad técnica: muchos vertidos no están asociados a un cauce nominal, por lo que la asignación a red fluvial requerirá estrategias de **proximidad** y tolerancias.

Esta capa estadística no es un “preámbulo”: define decisiones posteriores (CRS métrico, umbrales, robustez ante outliers) y evita que el análisis espacial herede errores silenciosos.

4.3 Preparación geoespacial: geometría, CRS y visualización

4.3.1 De tabla a GeoDataFrame (UTM → geometría)

El censo incluye coordenadas UTM (`UTM_X` , `UTM_Y`). Para operar con distancias reales (metros) y aplicar clustering basado en proximidad, se crea un `GeoDataFrame` con puntos y un CRS proyectado:

- **EPSG:32630 (UTM Zona 30N)** cuando las coordenadas están en UTM huso 30.
- La elección es crítica: algoritmos como DBSCAN usan distancia euclídea; trabajar en lat/lon introduce escalas inconsistentes.

Código mínimo:

```
gdf = gpd.GeoDataFrame(  
    df,  
    geometry=gpd.points_from_xy(df['UTM_X'],  
df['UTM_Y']),  
    crs="EPSG:32630"  
)  
  
print("Cálculos y geometría UTM generados  
correctamente.")
```

Salida esperada:

```
Cálculos y geometría UTM generados correctamente.
```

4.3.2 Contextualización con geometría fluvial (WKT → GeoPandas) y reproyección web

Para interpretar patrones (hotspots) respecto al río principal y tributarios, se incorporan capas lineales (p. ej., Tajo y afluentes) procedentes de CSV con geometría en WKT (`MULTILINESTRING(...)`). Se convierten a geometrías con `wkt.loads` y se gestionan CRS:

- CRS de origen típico para capas hidrográficas: **EPSG:25830** (ETRS89 / UTM huso 30).
- Para mapas web (Folium), se reproyecta a **EPSG:4326** (lat/lon), ya que los tiles web operan en ese sistema.

Esto permite superponer puntos (vertidos), líneas (río) y capas derivadas (clústeres/heatmaps) en visualizaciones interactivas, facilitando auditoría visual y comunicación de resultados.

4.4 Clustering espacial: detección de hotspots con DBSCAN

4.4.1 Por qué DBSCAN y qué aporta frente a alternativas

Para detectar agrupaciones sin imponer un número fijo de clústeres, se utiliza DBSCAN, en inglés **Density-Based Spatial Clustering** (Kumar, 2026). A diferencia de k-means, DBSCAN:

- identifica clústeres de forma arbitraria (no esféricos),
- separa **ruido** (etiqueta `-1`) de concentraciones,
- no requiere elegir k a priori, útil cuando la distribución espacial es irregular.

La condición indispensable es operar en un CRS métrico (UTM), coherente con los parámetros `eps` en metros.

4.4.2 Dos parametrizaciones complementarias (5 km vs 2 km)

En el material aparecen dos configuraciones, que no son contradictorias sino **dos escalas de análisis**: 1) **Escala amplia (5 km)**: útil para detectar agrupaciones regionales y patrones administrativos o de cuenca.

```
coords = df[['UTM_X', 'UTM_Y']].values
db = DBSCAN(eps=5000, min_samples=3,
algorithm='ball_tree').fit(coords)
gdf['cluster_id'] = db.labels_
```

- **eps=5000** (5 km), **min_samples=3**.
- Más sensible a agrupaciones “suaves” y menos exigente en densidad.

2) **Escala local (2 km)**: orientada a hotspots más compactos (potenciales “zonas de sacrificio” por presión acumulada).

```

coords = df[['UTM_X', 'UTM_Y']]
db = DBSCAN(eps=2000, min_samples=5).fit(coords)
df['spatial_cluster'] = db.labels_

cluster_analysis = (
    df[df['spatial_cluster'] != -1]
    .groupby('spatial_cluster')
    .agg({
        'VOLUMEN_m3_ano': 'sum',
        'TITULAR': 'count',
        'MUNICIPIO': lambda x: x.mode()[0]
    })
    .sort_values('VOLUMEN_m3_ano', ascending=False)
)

```

- **eps=2000** (2 km), **min_samples=5**.
- Más conservador: exige densidad para declarar un hotspot.

Recomendación práctica: usar ambas escalas para comparar estabilidad de clústeres (Soriano, 2024). Si un hotspot aparece en 5 km pero se fragmenta en 2 km, suele indicar una conurbación amplia con subfocos; si persiste en 2 km, es un candidato fuerte para priorización.

4.5 Distribución del impacto: Curva de Lorenz, Gini y priorización (TIERs)

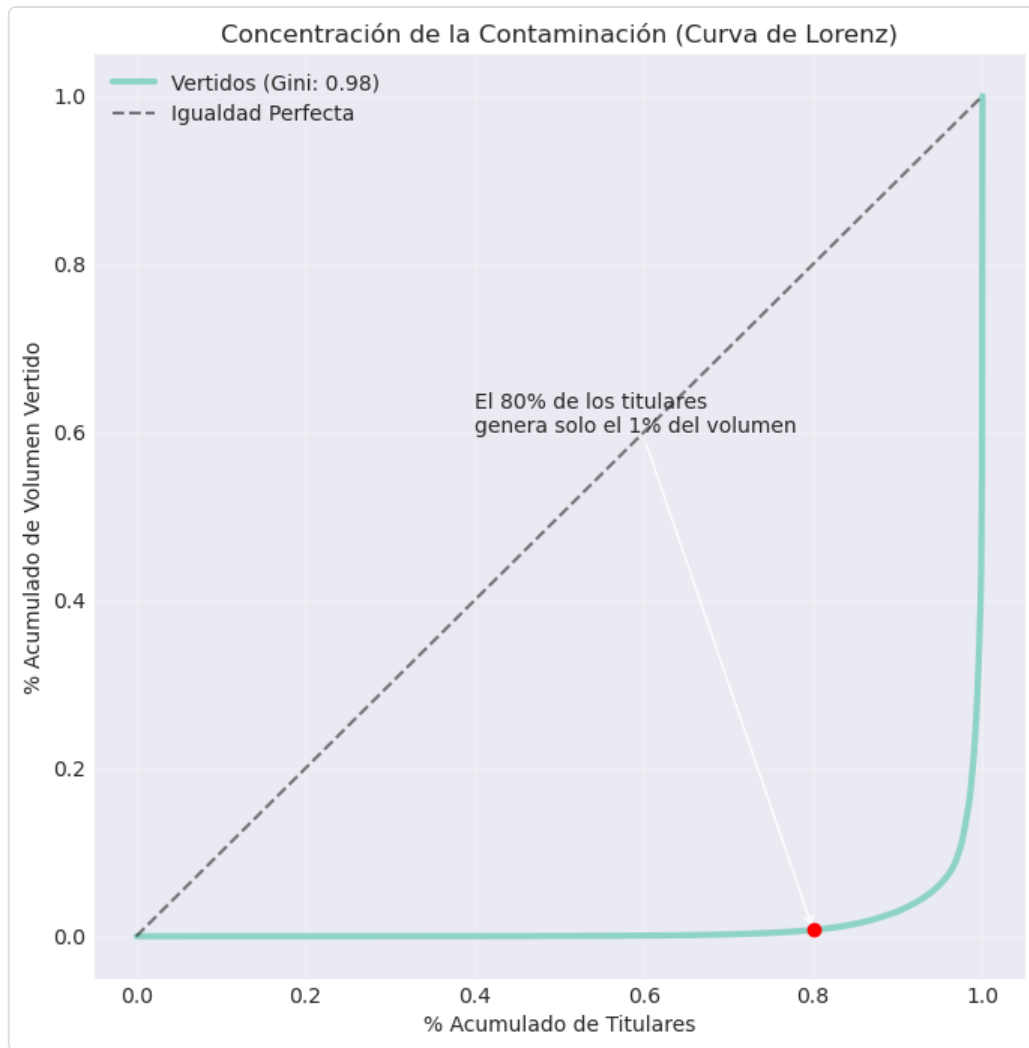
4.5.1 Motivación: ¿muchos aportan poco o pocos aportan mucho?

El conteo de autorizaciones no equivale a impacto. Para orientar vigilancia e inspecciones, interesa saber si el volumen vertido está:

- relativamente **repartido** entre muchos titulares, o
- **concentrado** en unos pocos (lo que habilita estrategias tipo Pareto).

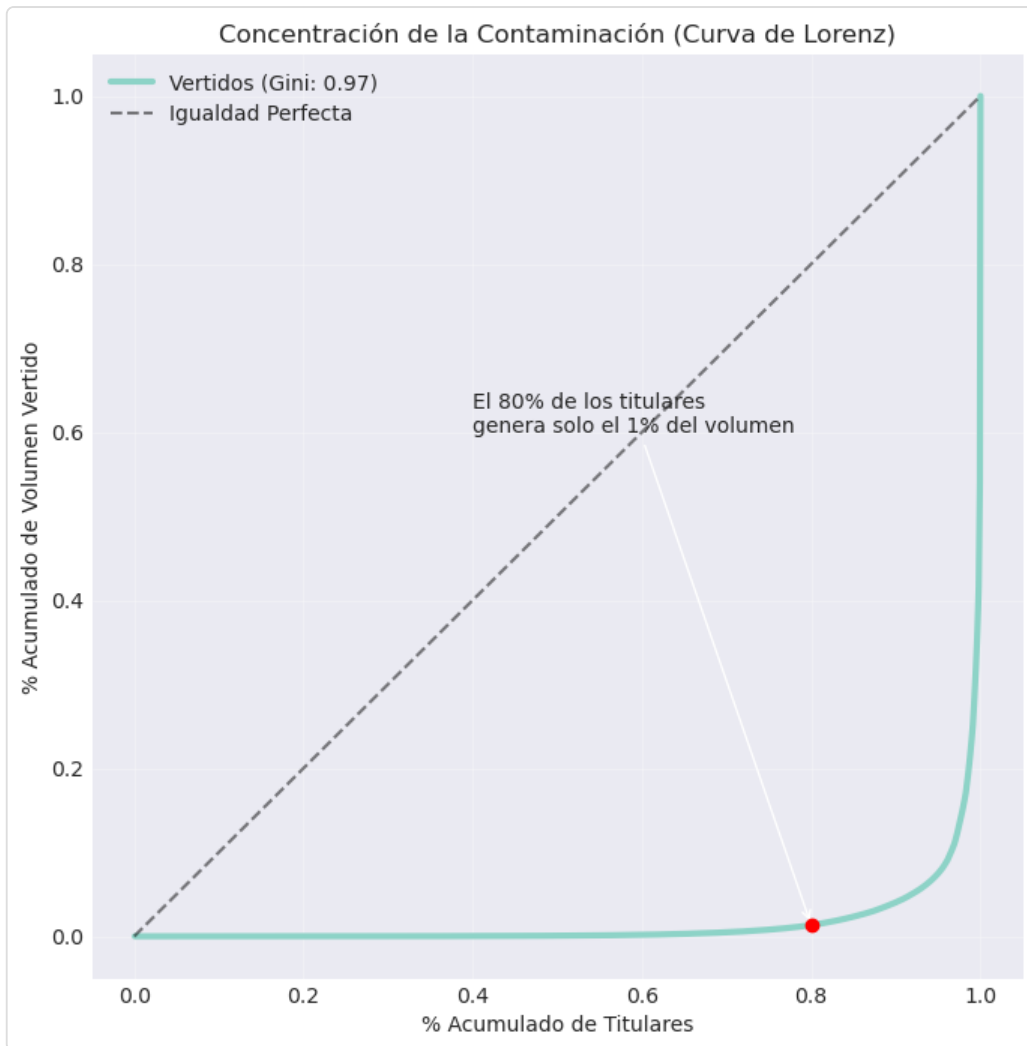
Aquí se emplean la **Curva de Lorenz** y el **coeficiente de Gini** (Vietri & Del Duca, 2023), para cuantificar desigualdad y sustentar una clasificación estratégica (TIERs) de fuentes contaminantes. Visualmente, la curva observada es extremadamente desigual (forma “palo de hockey”), como se aprecia en las siguientes gráficas:

.



[Figura 3.]²

2. Curva de Lorenz de concentración de contaminación (Gini $\approx 0,98$). Muestra que el 80% de los titulares genera apenas ~1% del volumen vertido, dejando ~99% del volumen en el 20% restante.

[Figura 4.]³

4.5.2 Cálculo de Gini desde Lorenz (integración numérica)

El Gini se deriva del área bajo la curva de Lorenz. En el notebook se calcula integrando el acumulado normalizado; se recomienda `np.trapezoid` frente a `np.trapz` (deprecado):

3. Curva de Lorenz de concentración de contaminación mostrando desigualdad extrema: alrededor del 80% de los titulares representan solo el ~1% del volumen total, mientras que el 20% restante representa el ~99%; la curva (Gini: 0,97) se sitúa muy por debajo de la línea de igualdad.

```
def plot_lorenz_curve(df, col_volumen):
    # Filtramos ceros y ordenamos
    vols = df[df[col_volumen] > 0]
    [col_volumen].sort_values().values

    # Acumulados normalizados
    cum_vols = np.cumsum(vols)
    cum_vols_norm = cum_vols / cum_vols[-1]

    # Eje X: población acumulada
    n = len(vols)
    x = np.arange(1, n + 1) / n

    # Gini (aprox.): 1 - 2 * área bajo la curva de
    Lorenz
    area_bajo_curva = np.trapezoid(cum_vols_norm, x)
    gini = 1 - 2 * area_bajo_curva

    return gini
```

Decisiones de preparación de datos (críticas para no sesgar):

- **Excluir volúmenes 0:** incluir muchos ceros incrementa la “población” sin aportar volumen y puede exagerar la desigualdad.
- **Evaluar el efecto del tipo de titular:** por ejemplo, excluir personas físicas (`tipo_titular != 'F'`) para evitar comparaciones injustas entre particulares y grandes operadores.

El resultado (concentración extrema) se convierte en una regla de negocio: identificar “grandes contribuyentes” y cruzarlos con hotspots espaciales para priorizar vigilancia donde coinciden **densidad y alto volumen agregado**.

4.6 Topología hidrológica mínima: eje del río, asignación de sensores y relaciones aguas abajo

4.6.1 Objetivo: convertir proximidad en relaciones operativas

Una vez que existen (a) puntos de vertido con clusters y (b) una noción de impacto (Lorenz/Gini), el siguiente paso es conectar el mundo geoespacial con el mundo de sensores: construir relaciones del tipo “este vertido podría ser observado por este sensor aguas abajo”. Esto habilita:

- análisis de cobertura (qué sensores “vigilan” qué zonas),
- priorización (si un sensor asociado a un hotspot detecta anomalía, elevar severidad),
- construcción de un grafo dirigido (p. ej., Neo4j) para consultas upstream/downstream.

4.6.2 Descarga y preparación del eje fluvial (OSM + Overpass)

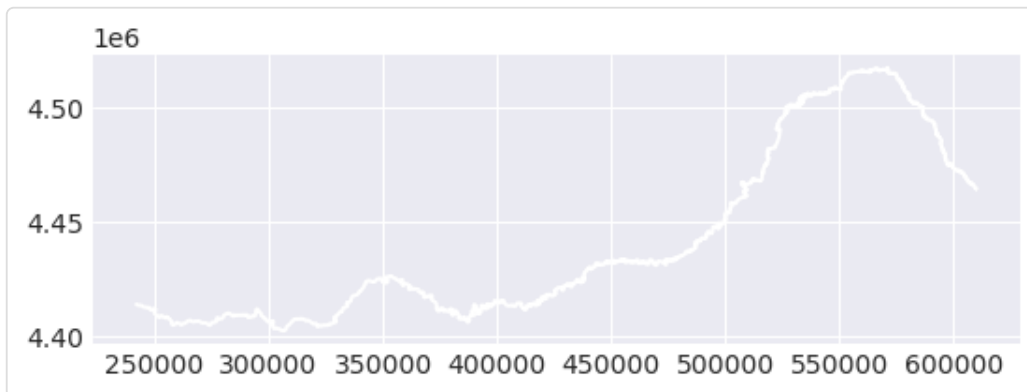
Para digitalizar el eje del río (ej. Tajo) se consulta Overpass restringiendo el área con una **Bounding Box** para evitar ruido (p. ej., tramos fuera del área de interés): `(39, -6, 41, -1.5)`. Se solicitan ways con `waterway=river` y `name="Río Tajo"`. Dos pasos técnicos son determinantes:

- **Unión de tramos:** OSM devuelve segmentos; se combinan con `linemerge(MultiLineString(...))` para obtener una geometría más continua.
- **Reproyección:** de EPSG:4326 a EPSG:32630 para distancias coherentes en metros (`distance()`, `project()`).

Salida de control:

Tajo descargado y reproyectado a UTM 30N.
 Longitud total del tramo capturado: 716.51 km

La confirmación visual del trazado aparece en la siguiente gráfica:



[Figura 5.]⁴

4.6.3 Asignación de sensores al río por proximidad (con umbral)

Con un diccionario de geometrías fluviales `geo_rios: {nombre_rio -> LineString/MultiLineString}`, cada sensor (con `utm_x`, `utm_y`) se asigna al río más cercano:

- se calcula `dist = pt.distance(geom)` para cada río candidato;
- se selecciona el mínimo;
- se aplica un **umbral de pertenencia**: si `dist < 3000 m`, el sensor se considera asociado a ese río (tolerancia a error de coordenadas y a simplificación geométrica).

4. Gráfico lineal de una métrica frente a un índice de 250.000 a 600.000. El valor se mantiene alrededor de 4,40 al principio, sube a un pico cerca de 4,50 alrededor de 560.000–570.000, y luego desciende a unos 4,45 para 600.000.

Además, se calcula la posición longitudinal del sensor sobre el eje:

- `km_rio = geom.project(pt)` devuelve la distancia (m) desde el inicio del río hasta la proyección ortogonal del punto sobre la línea.

Este escalar permite **ordenar sensores** a lo largo del río y, por extensión, definir relaciones aguas arriba/abajo de forma determinista.

4.6.4 Hacia la asignación vertido → sensor aguas abajo

Con (1) vertidos como puntos UTM, (2) sensores proyectados al eje del río y (3) una coordenada longitudinal (`project`) comparable, se puede construir una relación dirigida:

- para cada vertido, identificar sensores con `km_rio` mayor (aguas abajo) y seleccionar el más cercano según criterio (distancia hidráulica o combinación con distancia euclídea);
- almacenar distancias:
- **hidráulica** (a lo largo del río, diferencia de `project`),
- **euclídea** (línea recta).

El resultado son aristas tipo **AGUAS_ABAJO** listas para alimentar un grafo y conectar, de forma trazable, patrones espaciales (hotspots), prioridad por impacto (TIERs) y eventos en series temporales (anomalías detectadas por sensores).

5. Detección de anomalías y valores atípicos

5.1 Detección de anomalías y outliers: un enfoque por capas para calidad de datos

La suite aborda la detección de anomalías como un **problema de calidad de datos previo y complementario** a cualquier análisis posterior (modelos, imputación avanzada, reparación de series, auditoría). La idea transversal es que **no existe una única prueba suficiente**: se combinan capas que capturan tanto errores “duros” (instrumentales o de transmisión) como patrones sutiles (degradación del sensor, inconsistencias contextuales o “no naturalidad” estadística en datos administrativos). Este diseño reduce falsos positivos y permite priorizar revisiones humanas donde realmente aportan valor. En la práctica, el pipeline se organiza en tres grandes familias de señales:

- **Outliers puntuales** (picos, saturaciones, ceros espurios): típicos en sensores.
- **Anomalías contextuales multivariantes** (combinaciones improbables de variables): comunes en censos y registros con componente espacial/categoría.
- **Anomalías estructurales** (pérdida de dinámica/variabilidad, “flatlines”): fallos de sensor que pueden parecer “plausibles” si solo se miran rangos.

A continuación se integran las técnicas principales y cómo se conectan entre sí.

5.2 Capa base: control de outliers para regular sensibilidad antes de modelos avanzados

Antes de imputar, reparar o aplicar detectores más sofisticados, se aplica una capa de outliers con un objetivo operativo claro: **eliminar errores evidentes sin borrar eventos raros reales** (lluvias, vertidos puntuales, cambios operacionales). Esta capa actúa como “filtro de seguridad” para que etapas posteriores (p. ej., LOF, Isolation Forest contextual, detección de flatlines) no se saturan de falsos positivos (Yepes Piqueras, 2022).

5.2.1 Umbrales Sigma (Z-Score) como análisis de sensibilidad

Una herramienta útil para calibrar la agresividad del filtrado es el barrido de umbrales Sigma sobre Z-Score. En un dataset de **278,037 registros**, se cuantifica cuántas filas quedarían marcadas como outliers al variar el corte, mostrando la tensión clásica:

- Umbrales bajos (p. ej., **Sigma 3**) son estrictos: pueden eliminar valores raros pero reales.
- Umbrales altos (p. ej., **Sigma 10**) son laxos: solo capturan errores catastróficos.

Resultados ilustrativos del barrido:

- **Sigma 1.0** → 46,677 registros (16.7881%)
- **Sigma 2.0** → 5,680 registros (2.0429%)
- **Sigma 3.0** → 1,062 registros (0.3820%)
- **Sigma 6.0** → 157 registros (0.0565%)

- A partir de $\sim \text{Sigma } 9.7 \rightarrow 0$, coherente con el máximo Z observado.

Se verifican además extremos globales:

- Z-Score máximo: 9.6959
- Z-Score mínimo: -9.6959

Esto confirma la existencia de valores altamente extremos (errores potenciales o eventos muy atípicos) y justifica inspección dirigida. Como criterio operativo conservador —alineado con “no penalizar eventos reales”— se adopta $\text{Sigma} = 6$ como corte “generoso” en el flujo que desemboca en el Flatline cleaner.

5.2.2 De Z-Score global a rolling Z-Score: robustez temporal y estacionalidad corta

El análisis anterior es global (una foto agregada), pero en producción se prefiere un filtrado más robusto que respete la estructura temporal:

- **Z-Score en ventana móvil** (típicamente 4 días = 96 horas) para adaptarse a estacionalidad corta y episodios meteorológicos.
- **Transformación $\log_{1p}$** para variables de cola pesada (aprox. log-normales), estabilizando varianza y evitando que valores altos “normales” parezcan outliers.
- **Respeto de batches**: no mezclar estadísticas entre periodos desconectados (huecos, reinicios), evitando sesgos por discontinuidades.
- **Corte conservador** (p. ej., 6 sigmas) para centrarse en errores claros.

Esta capa “limpia lo obvio” y deja el resto a detectores que incorporan contexto, estructura y explicación.

5.3 Sensores: detección de anomalías estructurales (flatline) con enfoque multinivel

En sensores, muchos fallos no se manifiestan como picos extremos, sino como **pérdida de dinámica**: el dato puede estar dentro de rango, pero deja de representar el fenómeno. Por ello, la detección se plantea como un **sistema por capas** que captura desde congelaciones triviales hasta estancamientos sutiles y fallos dependientes del contexto horario.

5.3.1 Enfoque multinivel: de fallos obvios a fallos “contextuales”

El principio es encadenar validaciones complementarias para aumentar confianza:

- **Nivel 1 (repetición exacta)**: errores duros (software/transmisión, congelación del valor).
- **Nivel 2 (pérdida de variabilidad)**: señal “viva” en apariencia, pero sin ruido/variación natural (p. ej., biofouling).
- **Nivel 3 (inconsistencia con ciclo diario)**: flatlines en periodos donde debería haber cambio (amanecer/atardecer), usando derivadas y contexto horario.

Este enfoque no compite con la detección de outliers puntuales; la completa: mientras el Z-Score/rolling Z-Score captura picos o valores imposibles, el detector de flatline captura **degradación temporal**.

5.3.2 Nivel 1 — Repetición exacta (flatline “tonto”)

Qué detecta: el sensor envía exactamente el mismo valor flotante durante un periodo prolongado. **Lógica**: contar rachas consecutivas de igualdad exacta. Si `valor_t == valor_{t-1}`

== ... durante, por ejemplo, **60 minutos**, se marca como fallo. **Justificación:** en magnitudes físicas con ruido y decimales, una igualdad exacta sostenida es extremadamente improbable; suele indicar congelación del dato o repetición en la cadena de adquisición. **Ventajas y límites:**

- Ventaja: muy preciso y barato computacionalmente.
- Limitación: no detecta estancamientos “casi constantes” (p. ej., $7.01 \leftrightarrow 7.02$).

5.3.3 Nivel 2 — Pérdida de variabilidad (señal “sin vida”)

Qué detecta: la señal no queda clavada en un único número, pero su variabilidad cae por debajo de lo esperable. **Lógica:** medir dispersión en ventana rodante (desviación estándar u opciones robustas):

- Calcular **σ_{rolling}** en una ventana (p. ej., **6–12 horas**).
- Marcar si **$\sigma_{\text{rolling}} < \varepsilon$** durante **X horas**, donde **ε** se define por la **resolución real** del sensor o un ruido mínimo aceptable.

Caso típico: sensores ópticos (clorofila) con biofouling: variables naturalmente ruidosas cuya dispersión colapsa cuando el sensor deja de responder. **Diferencia clave vs outliers:** aquí no se buscan picos aislados, sino **colapso de dispersión**; por eso convive con rolling Z-Score (eventos) y no lo sustituye.

5.3.4 Nivel 3 — Inconsistencia con el ciclo diario (contexto horario)

Aunque no siempre basta con “variabilidad baja”, hay escenarios donde el contexto manda: si el sistema sabe que en ciertas franjas (amanecer/atardecer) debería haber cambios, entonces un estancamiento en esos periodos es más sospechoso. Este nivel

introduce **contexto temporal** (y típicamente derivadas o cambios esperados) para detectar flatlines “inteligentes”: datos que parecen razonables pero fallan justo cuando deberían moverse.

5.4 Registros de vertidos: anomalías por “no naturalidad” y outliers contextuales geoespaciales

En el censo de autorizados a verter, la anomalía no siempre significa “valor extremo”. A menudo interesa detectar señales de **no naturalidad estadística** (posible redondeo sistemático, estimación manual o generación por plantilla) y, en paralelo, **outliers contextuales**: volúmenes o ubicaciones raras en combinación con otras variables.

5.4.1 Auditoría forense de números: Benford y dígito terminal

Estas pruebas no pretenden “probar fraude”, sino generar **señales cuantitativas** que prioricen revisión metodológica (origen del dato, trazabilidad, procedimiento de estimación).

5.4.1.1 Ley de Benford (primer dígito)

Qué mide: en magnitudes reales que abarcan varios órdenes de magnitud, el primer dígito significativo no es uniforme (1 aparece ~30%, 9 ~4.6%). Desviaciones fuertes sugieren **redondeo sistemático, estimación manual o datos inventados**. **Implementación crítica:** extracción del primer dígito significativo y contraste Chi-cuadrado:

```
def benford_audit(series):
    first_digits =
series.astype(str).str.lstrip('0').str[0].astype(int)
    first_digits = first_digits[first_digits > 0]

    obs_counts =
first_digits.value_counts().sort_index()
    total = obs_counts.sum()
    expected_probs = np.log10(1 + 1 / np.arange(1, 10))
    exp_counts = expected_probs * total

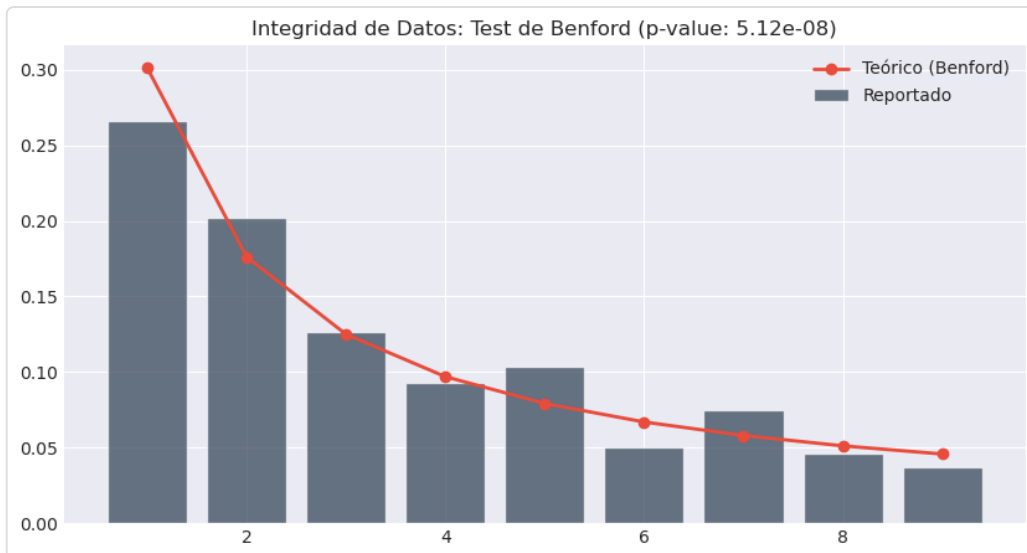
    chi2, p_val = chisquare(obs_counts,
f_exp=exp_counts)
    return p_val

p_value = benford_audit(df['VOLUMEN_m3_ano'])
print(p_value)
```

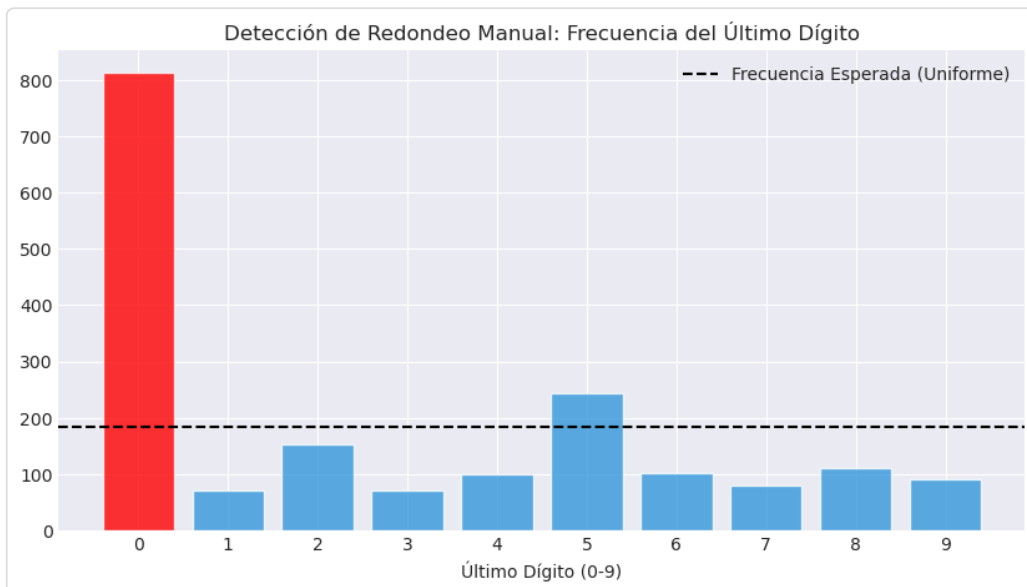
Resultado clave: para 1835 registros operativos se obtiene un **p-valor extremadamente bajo**:

```
INFO: p-valor 5.118926842628549e-08
```

La desviación se aprecia en la comparación visual de frecuencias observadas vs teóricas:



[Figura 6.]⁵



[Figura 7.]⁶

5. Gráfico de barras de las frecuencias observadas del primer dígito (azul) para los dígitos 1–9, con la distribución teórica de Benford (línea roja) superpuesta. Las frecuencias observadas caen de ~0,26 para el dígito 1 a ~0,04–0,05 para el dígito 9, desviándose de la ley de Benford; el valor p de la prueba es 5,12e-08, indicando una desviación significativa.

6. Gráfico de barras de las frecuencias observadas del último dígito (0–9) en redondeo manual. El dígito 0 domina (~810 apariciones), mientras que otros dígitos son mucho más bajos (aprox. 70–240, con el 5 en ~240); la línea de expectativa uniforme está en unos 200, indicando una fuerte desviación hacia el 0.

Interpretación: no es evidencia concluyente por sí sola, pero sí una señal fuerte de que parte de los volúmenes **no se comporta como medición instrumental**.

5.4.1.2 Análisis del dígito terminal (último dígito)

Qué mide: en mediciones reales, el último dígito (0–9) tiende a ser aproximadamente uniforme; en carga humana aparecen sesgos hacia 0 y 5. **Resultado clave:** se observa una **sobre-representación del dígito ‘0’ de +342.5%**, indicio de redondeo masivo:

```
Sobre-representación del dígito '0': +342.5% (Indicio  
de redondeo masivo)
```

La asimetría se visualiza en

. De nuevo, la lectura es operativa: priorizar revisión de fuentes y procedimientos, no dictaminar causalidad automáticamente.

5.4.2 Outliers multivariantes: magnitud + geografía con Isolation Forest

Cuando el objetivo es detectar registros atípicos por **combinación de magnitud y contexto espacial**, se construye un espacio de características que haga comparables los volúmenes y capture “dónde ocurre”:

- **UTM_X, UTM_Y:** contexto espacial.
- **log_vol:** volumen anual en escala logarítmica para manejar **colas pesadas** (frecuente en magnitudes ambientales).

Con los inliers se define una **Línea Base Operativa**: el perfil estándar de vertidos contra el que evaluar desviaciones futuras,

evitando confundir grandes vertedores esperables con casos realmente sospechosos. **Isolation Forest** permite detectar:

- volúmenes anómalos para su zona,
- registros geográficamente aislados aunque el volumen no sea extremo,
- combinaciones improbables (p. ej., tipología de titular vs magnitud).

Configuración mínima:

```
from sklearn.ensemble import IsolationForest

features = df[['UTM_X', 'UTM_Y', 'log_vol']].fillna(0)

# Contaminación ~1% para centrarse en extremos
iso_forest = IsolationForest(n_estimators=100,
                              contamination=0.01, random_state=69420)

df['anomaly_score'] = iso_forest.fit_predict(features)
# -1 anomalía, 1 normal
anomalies = df[df['anomaly_score'] ==
-1].sort_values('VOLUMEN_m3_ano', ascending=False)
```

Lectura operativa: `anomalies` es una cola priorizada para auditoría/contraste con expediente/inspección, no “fraude confirmado”. Para reducir falsos positivos, puede aplicarse filtrado por tipología (p. ej., excluir **ayuntamientos** si dominan por escala y sesgan el ranking).

5.4.3 Anomalías contextuales locales: LOF y explicación por vecinos

Si el interés es “extraño para su contexto inmediato” (por ejemplo, volumen raro para ese tipo de vertido o medio receptor), **Local**

Outlier Factor (LOF) es especialmente útil porque compara **densidades locales**: un punto es anómalo si su vecindario es denso pero él queda relativamente aislado. Decisiones de configuración destacadas:

- métrica **manhattan** : suele funcionar mejor que **euclidean** con features mixtas (numéricas + codificaciones),
- **n_neighbors=20** : tamaño del contexto local,
- **contamination=0.01** : ~1% como punto de partida (a calibrar).

En el censo se reportan **12 anomalías** interpretadas como volúmenes inusuales para combinaciones como MEDIO_RECEPTOR, NATURALEZA_VERTIDO y VOLUMEN_m3_ano:

```
=== ANOMALÍAS DETECTADAS (12) ===
Interpretación: Vertidos con un volumen extraño PARA
ESE TIPO de vertido o ESE RÍO.
```

5.4.3.1 Interpretabilidad práctica: “por qué” mediante comparación con k vecinos

Una ventaja operativa de LOF es su explicabilidad natural: se puede justificar una marca comparando la anomalía con sus **k vecinos** más similares en el espacio de features. Esto permite señalar si la rareza viene de categorías (es el “patito feo” del vecindario) o de magnitud (volumen desalineado con la media local). Código de explicación basado en **kneighbors** :

```

def explicar_anomalia_lof_vecinos(modelo_lof,
matriz_features, df_original, indice_anomalia_iloc):
    """
        Toma una anomalía, busca sus k-vecinos en el
        espacio matemático y explica las diferencias.
    """
    anomalia_data =
df_original.iloc[indice_anomalia_iloc]
    vector_anomalia =
matriz_features[indice_anomalia_iloc].reshape(1, -1)

    distancias, indices_vecinos =
modelo_lof.kneighbors(vector_anomalia)
    indices_vecinos = indices_vecinos[0]
    vecinos_df = df_original.iloc[indices_vecinos]

    print(f"\n --- ANÁLISIS FORENSE:
{anomalia_data['TITULAR']}
({anomalia_data['MUNICIPIO']}) ---")
    print(f"    (Comparado contra sus {len(vecinos_df)}
vecinos más similares matemáticamente)")

    cols_cat = ['MEDIO_RECEPTOR', 'NATURALEZA_VERTIDO']
    for col in cols_cat:
        val_anomalo = anomalia_data[col]
        moda_vecinos = vecinos_df[col].mode()[0]
        pct_coincidencia = (vecinos_df[col] ==
val_anomalo).mean() * 100

        if val_anomalo != moda_vecinos:
            print(f"RAREZA EN {col}:")
            print(f"El vertido es '{val_anomalo}' pero
la moda en vecinos es '{moda_vecinos}'.")

```

```
print(f"Coincidencia con vecinos:
{pct_coincidencia:.1f}%")
```

5.5 Comparativa breve: cuándo usar cada técnica

- **Z-Score / rolling Z-Score (con `log1p` y batches)**: ideal para control de calidad rápido y conservador, especialmente para **outliers puntuales** en series temporales; sensible a la elección de ventana y umbral.
- **Detección de flatlines multinivel**: especializada en **fallos estructurales** (congelación o pérdida de dinámica) que no aparecen como extremos; complementa al filtrado por outliers.
- **Isolation Forest**: detección **global multivariante** (p. ej., geografía + magnitud); útil para priorizar colas de revisión cuando hay combinaciones improbables.
- **LOF**: detección **local/contextual** con ventaja de explicación por vecinos; especialmente útil cuando “lo raro” depende del vecindario (tipo de vertido, medio receptor, río).
- **Benford + dígito terminal**: auditoría forense de **naturalidad numérica**; no detecta “extremos” sino patrones de redondeo/plantilla y posibles sesgos de generación del dato.

Integradas, estas capas permiten pasar de “limpieza mínima segura” a “detección contextual explicable”, manteniendo el foco del proyecto: **mejorar la confiabilidad del dato sin destruir señales reales** y con trazabilidad suficiente para auditoría e intervención (Amat Rodrigo, 2020).

6. Transformación y exportación de datos

6.1 Transformación de datos: de la depuración al dataset analítico

La suite aborda un objetivo común: convertir fuentes heterogéneas (registros geoespaciales de vertidos, relaciones para grafo y grandes mediciones de sensores) en **datasets consistentes, reproducibles y listos para análisis** (clustering, detección de irregularidades y validación de series temporales). En la práctica, esto se logra encadenando tres tipos de transformaciones: (1) filtrado y selección de variables para reducir ruido, (2) preprocesamiento estadístico/categorico para modelado, y (3) normalización estructural para exportación y reutilización.

6.1.1 Filtrado de calidad: excluir administración pública para mejorar comparabilidad

En el censo de autorizados a verter, un primer paso de transformación consiste en **eliminar registros cuyo comportamiento puede sesgar el análisis**. En particular, se excluyen los titulares de administración pública (`tipo_titular == 'A'`) para centrar el estudio en titulares no públicos y evitar mezclar patrones potencialmente distintos (infraestructuras públicas vs. actividades privadas). Este filtrado reduce ruido y

mejora la comparabilidad en etapas posteriores (p. ej., clustering o detección de irregularidades).

```
# Quitamos a la admin pública  
tmp = df[df['tipo_titular'] != 'A'].copy()
```

El uso de `.copy()` no es un detalle menor: evita efectos colaterales en pandas (como `SettingWithCopyWarning`) y garantiza que las transformaciones posteriores se aplican sobre un DataFrame independiente, manteniendo el pipeline más predecible y reproducible.

6.1.2 Preparación del dataset: selección de columnas y tipado numérico/categórico

Tras el filtrado, se construye un subconjunto con las columnas relevantes para el análisis (`columnas_analisis`) y se separan variables **numéricas** y **categóricas** según su tipo. Esta separación es clave porque cada familia de variables requiere transformaciones distintas:

- **Numéricas:** escalado para homogeneizar magnitudes y estabilizar modelos sensibles a escala.
- **Categóricas:** codificación para preservar semántica sin imponer orden artificial.

```

datos = tmp[columnas_analisis].copy()

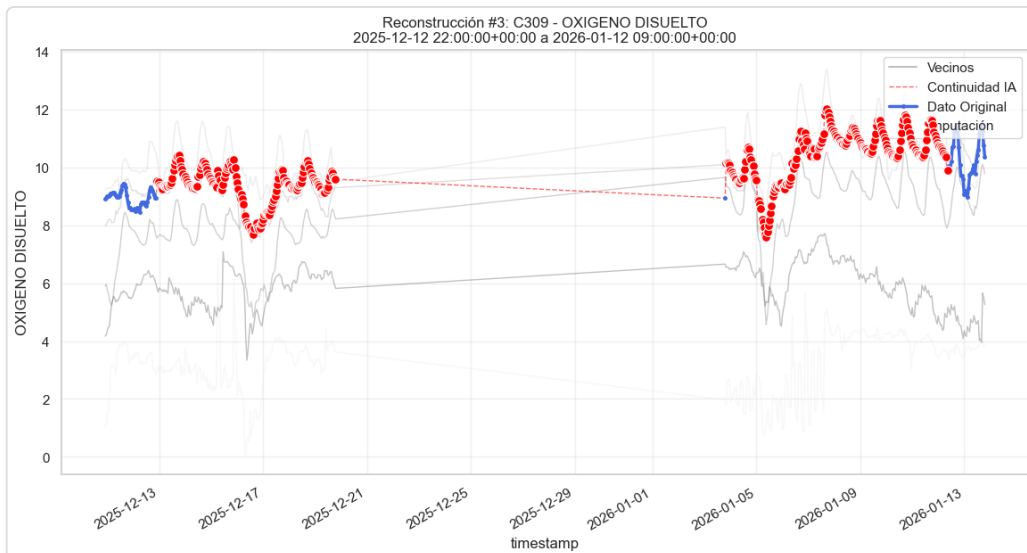
col_numericas = datos.select_dtypes(include=['float64',
'int64']).columns.tolist()
col_categoricas =
datos.select_dtypes(include=['object',
'category']).columns.tolist()

print(f"Configuración Detectada:")
print(f"    - Numéricas (Se escalarán):
{col_numericas}")
print(f"    - Categóricas (Se harán One-Hot):
{col_categoricas}")

```

En un caso típico, el pipeline detecta `log_vol` como numérica principal (frecuente cuando el volumen original está sesgado y se ha transformado logarítmicamente) y variables como `MEDIO_RECEPTOR` o `NATURALEZA_VERTIDO` como categóricas, lo que permite tratarlas sin perder su significado. Si en el notebook se muestran celdas de salida con esta configuración, conviene referenciarlas como evidencia de trazabilidad, por ejemplo:

.



[Figura 8.] Reconstrucción de Oxígeno Disuelto en C309

6.1.3 Preprocesamiento unificado: escalado + One-Hot con ColumnTransformer

Para que el preprocesamiento sea **consistente y reutilizable**, se define un `ColumnTransformer` que aplica en un único objeto:

- `StandardScaler` a columnas numéricas (media 0, desviación 1).
- `OneHotEncoder` a categóricas, usando `handle_unknown='ignore'` para robustez ante categorías nuevas (muy habitual al integrar datos de distintos periodos o fuentes).

```
preprocessor = ColumnTransformer(
    transformers=[
        ('num', StandardScaler(), col_numericas),
        ('cat', OneHotEncoder(handle_unknown='ignore',
                               sparse_output=False), col_categoricas)
    ]
)
```

Este enfoque “todo en uno” reduce errores operativos (por ejemplo, aplicar transformaciones distintas en entrenamiento vs. inferencia) y deja el dataset en un formato directamente consumible por algoritmos que requieren matrices numéricas densas. En flujos con múltiples notebooks y colaboradores, encapsular la lógica en un preprocesador explícito también facilita auditoría y repetición del proceso.

6.2 Consolidación: construir artefactos reutilizables y un “DataFrame maestro”

Una vez transformados los datos para análisis, el siguiente reto es **consolidar** resultados intermedios en artefactos compartibles. La consolidación cumple dos funciones: (1) actuar como “punto de control” para no recomputar pasos costosos y (2) asegurar que etapas downstream (por ejemplo, inyección/consulta en Neo4j o análisis de vecindad) parten de un dataset coherente.

6.2.1 Exportaciones intermedias a CSV como puntos de control

En el flujo geoespacial, es habitual exportar capas ya transformadas (incluyendo GeoDataFrames) a CSV para consumo posterior sin necesidad de repetir todo el pipeline:

```
# gdf_rio_utm.to_csv("/tmp/pycharm_project_992/data/
datasets/raw/linestring_tajo.csv")
```

Operativamente, conviene mantener rutas estables dentro de `data/datasets/` (por ejemplo `raw/` y `processed/`) y **versionar el esquema** (nombres de columnas, CRS si aplica, y convenciones de unidades). Esta disciplina reduce ambigüedades entre iteraciones y facilita colaboración, especialmente cuando distintos módulos (p.

ej., limpieza de series temporales o análisis de irregularidades) consumen estos outputs.

6.2.2 DataFrame maestro para Neo4j: concatenación de “capas” y control de integridad

Para preparar la carga en Neo4j, se construye un **DataFrame maestro de relaciones** (por ejemplo, `df_aristas_final`) que integra varias “capas de inteligencia”:

- **Física (agua):** conectividad hidrológica/aguas abajo.
- **Proximidad:** relaciones derivadas de cercanía o vecindad espacial.
- **Administración:** pertenencia territorial o titularidad (p. ej., municipio).

La técnica central es concatenar DataFrames de aristas ya normalizados (misma estructura de columnas clave) y aplicar una limpieza final crítica: **no permitir IDs nulos** en `source_id` y `target_id`. Esto evita relaciones huérfanas y fallos silenciosos al cargar el grafo o al ejecutar consultas de vecindad.

```

dfs_to_concat = [
    df_aristas_espaciales,
    df_titulares,
    df_municipios
]

df_aristas_final = pd.concat(dfs_to_concat,
                             ignore_index=True)

# Limpieza final de nulos en IDs por seguridad
df_aristas_final =
df_aristas_final.dropna(subset=['source_id',
                                'target_id'])

print(f"GRAFO COMPLETO GENERADO:
      {len(df_aristas_final)} relaciones")

```

Este paso conecta directamente con el objetivo global del proyecto: antes de detectar anomalías o irregularidades, se garantiza que la estructura de datos (tabular o grafo) sea consistente y verificable.

6.3 Exportación final: del grafo a un reporte tabular listo para análisis

Cuando el grafo ya está construido y enriquecido, el cierre del flujo prioriza la **portabilidad**: extraer un reporte tabular para que otros miembros del equipo puedan trabajar sin levantar Neo4j ni reproducir el entorno completo. El resultado es un CSV que funciona como interfaz “amigable” para auditorías, priorización de inspecciones y cruces con otras fuentes (incluyendo series temporales).

6.3.1 Asignación Vertido–Sensor: estrategia topológica con plan B geométrico

La exportación desde Neo4j se apoya en una consulta Cypher que, para cada nodo `(:Vertido)`, intenta asignar un `(:Sensor)` siguiendo dos enfoques complementarios:

- **Prioridad topológica (por agua):** busca un camino aguas abajo (`[ :AGUAS_ABAJO|FLUYE_HACIA*]`) hasta un sensor, evitando atravesar sensores intermedios para no “saltar” de sensor en sensor.
- **Plan B geométrico:** si no existe conexión hidrológica, asigna el sensor más cercano por distancia euclídea usando coordenadas UTM.

Además de propiedades del vertido y del sensor, el reporte incluye metadatos de control como método de asignación, distancia y número de saltos topológicos. Esta instrumentación es esencial para interpretar resultados: no es lo mismo una asignación por conectividad hidrológica que una por proximidad.

6.3.2 Aplanado de mapas y normalización de campos para evitar colisiones

Los resultados del driver de Neo4j suelen llegar como registros con **mapas/diccionarios** de propiedades. Para convertirlos en un dataset utilizable en pandas/BI se aplica un aplanado sistemático:

- Prefijo `v_` para propiedades del vertido.
- Prefijo `s_` para propiedades del sensor.
- Columnas explícitas de control: `METODO_ASIGNACION`, `DISTANCIA_M`, `SALTOS_GRAFO`.

También se normaliza `SALTOS_GRAFO` rellenando nulos con `0` (típicamente cuando la asignación fue por proximidad o no hubo camino topológico). El resultado es un CSV sin colisiones de nombres y listo para herramientas no orientadas a grafos. Si se

muestra la tabla de columnas generadas o conteos de filas, puede referenciarse como evidencia de extracción, por ejemplo:

#	target_sensor	neighbor_sensor	relation_type	distance_m	initial_weight
0	C302	C312	UPSTREAM	16292.36	1.0
1	C302	C320	UPSTREAM	70000.12	1.0
2	C302	C316	UPSTREAM	114616.08	1.0
3	C302	C329	DOWNSTREAM	9637.54	0.8
4	C302	C315	DOWNSTREAM	56949.85	0.8

.

6.4 Serialización eficiente: Parquet con categorías para grandes datasets de sensores

En datasets masivos (especialmente series de sensores), el cuello de botella suele ser I/O y tamaño en disco. Por eso, además de CSV como “checkpoint” humano, se adopta **Parquet** como formato de staging para pipelines de alto rendimiento, priorizando consistencia de tipos y compatibilidad downstream.

6.4.1 category + Parquet: Dictionary Encoding automático

Un patrón de optimización clave es convertir columnas repetitivas (texto o IDs) a `category` antes de escribir a Parquet (vía `pyarrow`). Esto habilita **Dictionary Encoding**:

- Se almacena un diccionario interno con valores únicos.
- Cada fila guarda un código entero que referencia el diccionario, evitando repetir cadenas.

Es especialmente útil en columnas típicas como `tag`, `sensor_id`, `metric`, `unit` o `batch`, donde la cardinalidad suele ser baja/moderada frente al número de filas.

6.4.2 Implementación recomendada y particionado por batch

El flujo operativo recomendado es: convertir columnas candidatas (idealmente sobre un DataFrame controlado), y exportar a Parquet con compresión y particionado cuando aplique (por ejemplo, por `batch`, que facilita lecturas selectivas y checkpoints por lote).

```
cols_to_convert = ['tag', 'sensor_id', 'metric',
                  'unit', 'batch']

for col in cols_to_convert:
    if col in clean_df.columns:
        clean_df[col] =
clean_df[col].astype('category')
    print("Guardando a Parquet con categorías
nativas...")

clean_df.to_parquet(
    OUTPUT_DIR,
    partition_cols=['batch'],
    engine='pyarrow',
    compression='snappy',
    index=False
)

print(f"Datos guardados en {OUTPUT_DIR}")
```

6.4.3 Comparativa breve: CSV vs Parquet en esta suite

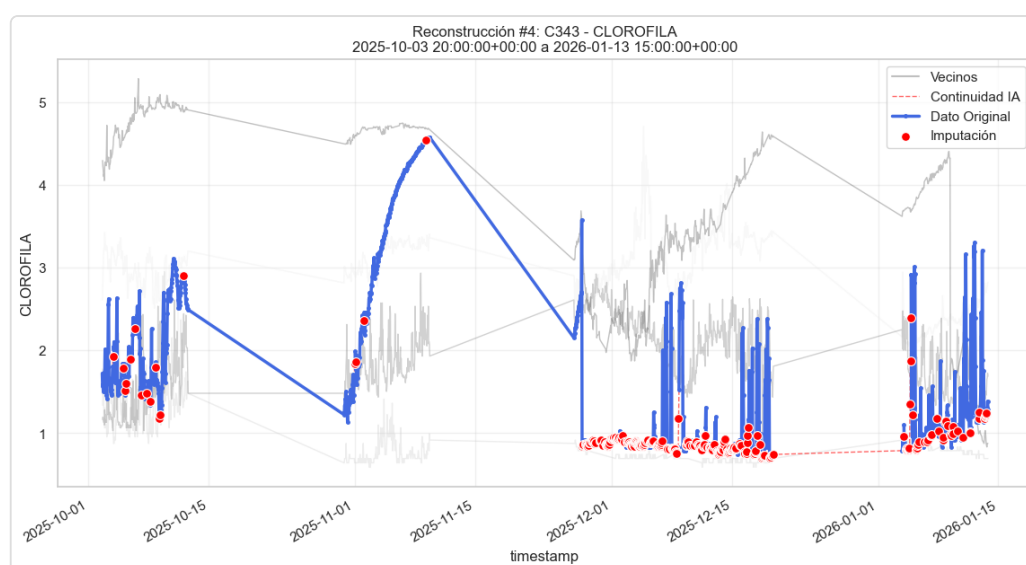
Ambos formatos conviven porque resuelven necesidades distintas:

- **CSV:** ideal como artefacto compartible “universal” (auditoría, revisión rápida, intercambio con perfiles no técnicos o BI). Se usa como punto de control tras transformaciones clave (capas geoespaciales, reportes desde Neo4j).
- **Parquet:** preferible para **volumen alto** y pipelines repetitivos como lectura rápida, compresión eficiente, tipado robusto, particionado (Alija, 2021). Es el formato natural para staging de mediciones de sensores y procesos posteriores (incluyendo detección de anomalías y limpieza temporal).

En conjunto, estas prácticas de transformación y exportación convierten el trabajo en notebooks, en un flujo reproducible: se depura y tipa el dato, se consolida en estructuras coherentes (tablas y grafo) y se serializa en artefactos que equilibran portabilidad (CSV) y rendimiento (Parquet).

7. Análisis y gestión de datos de sensores

7.1 Visión general del flujo: de contexto geoespacial a series temporales confiables



[Figura 9.] ⁷

7. Gráfico de serie temporal de Clorofila del 01-10-2025 al 15-01-2026, mostrando datos Originales (línea azul con marcadores), valores Imputados (puntos rojos), continuidad IA (línea roja discontinua) y una estimación de Vecinos (línea gris). Los datos Originales alcanzan un pico de 4,5–5,0 a mediados de noviembre y luego caen a ~1,0 a principios de diciembre, mientras que los valores imputados rellenan huecos entre 0,8 y 3,0 y la continuidad de la IA se mantiene cerca de 1,0–1,2.

La gestión y el análisis de datos de sensores ambientales requieren tratar simultáneamente dos dimensiones: (1) **el contexto espacial/administrativo** que explica qué puede afectar a cada punto de medida y (2) **la calidad temporal de las series** que determina si lo observado es un fenómeno real o un artefacto de adquisición. El pipeline unificado aborda ambos frentes para habilitar análisis robustos y detección de anomalías con menos falsos positivos. En primer lugar, se construye una capa de **relaciones vertido–sensor** (y vertido–titular/municipio) para poder responder preguntas operativas del tipo “¿qué fuentes potenciales afectan a este sensor?” y “¿qué sensores vigilan un área con alta presión de vertidos?”. A continuación, sobre los datos temporales, se establecen **criterios de normalidad y calibración** (baseline), se inspeccionan **anomalías físicas** (valores imposibles) y se aplica una **segmentación por rupturas** que evita mezclar periodos no comparables. Finalmente, se calculan **deltas y volatilidad por sensor** dentro de segmentos coherentes, preparando el terreno para módulos de limpieza (p. ej., flatlines) e imputación avanzada.

7.2 Contextualización: asignación de vertidos a sensores y relaciones administrativas

7.2.1 Grafo consultable: por qué conectar vertidos, sensores y administración

Para que la analítica temporal tenga significado, es esencial poder cruzar series con el “entorno” que las condiciona. La etapa de integración del censo de autorizaciones/vertidos con la red de sensores y entidades administrativas (titulares, municipios)

transforma tablas aisladas en una estructura consultable tipo grafo, útil para:

- **Atribuir vertidos a un sensor representativo** cuando no existe una relación hidrológica explícita.
- Habilitar consultas de **vecindad/topología** (“sensores cercanos”, “vertidos asociados a este sensor”), que luego se reutilizan en calidad de datos y detección de anomalías (por ejemplo, como proveedor de vecinos en Neo4j para análisis posteriores).
- Diagnosticar patrones sistemáticos por **titular o municipio**, facilitando priorización y análisis exploratorio.

El resultado práctico es una ruta de consulta consistente: **vertido** → **sensor cercano** → **vecindario de sensores** → **series temporales**, y también **vertido** → **titular/municipio** para análisis administrativo.

7.2.2 Rescate por proximidad: asignación del sensor más cercano

Cuando no se puede mapear un vertido mediante reglas hidrológicas (por falta de geometría, correspondencia río–estación, etc.), se aplica un rescate por proximidad usando una matriz de distancias `dist_matrix` (vertidos × sensores):

- `idx_nearest = dist_matrix.argmin(axis=1)` selecciona el sensor más cercano por vertido.
- `dist_nearest = dist_matrix.min(axis=1)` obtiene la distancia mínima asociada.

Con ello se construyen aristas espaciales `rels_rescate` con campos clave:

- `source_id` : vertido (`REF_EXPEDIENTE`)
- `target_id` : sensor (`id_estacion`)
- `distancia_euclidea` : distancia mínima

- `tipo_relacion`: `ASIGNADO_TEORICO` (heurístico, por cercanía)
- Campos no aplicables: `distancia_hidraulica` (desconocida) y `detalle_rio` marcado como `NO_MAPEAR`

Estas aristas se concatenan con relaciones hidrológicas existentes (`df_rels_hidro`) para producir un master unificado (`df_aristas_espaciales`) que conserva la semántica de la conexión:

```
Rescatados 1698 vertidos asignados por proximidad.
```

```
Total Aristas Espaciales: 1835
```

```
tipo_relacion
```

```
ASIGNADO_TEORICO      1698
```

```
AGUAS_ABAJO           137
```

Mantener `tipo_relacion` permite **filtrar o ponderar** conexiones en análisis posteriores: `AGUAS_ABAJO` suele ser más “fuerte” (hidrológica/topológica), mientras que `ASIGNADO_TEORICO` es una aproximación geométrica útil para cobertura.

7.3 Métricas de presión: carga y “estrés” esperado por sensor

7.3.1 Qué mide el “estrés” y por qué importa para anomalías

Una vez que vertidos y sensores están conectados, se puede estimar qué sensores están sometidos a mayor “presión” potencial de contaminación, entendida como el **volumen/carga de vertidos**

que podría afectar al tramo vigilado. Esta métrica actúa como puente entre:

- La asignación geoespacial (fuentes y contexto por municipio/cuenca),
- La preparación masiva de datos (para incorporar estas métricas como features),
- Y la interpretación de fallos temporales (p. ej., un flatline en una zona de alta carga es más sospechoso que en una zona estable).

En términos de grafo, se modelan nodos como `(:Vertido)` (con `volumen`, coordenadas UTM) y `(:Sensor)` (con `name`, `rio`, UTM), y relaciones hidrológicas/topológicas `[ :AGUAS_ABAJO | FLUYE_HACIA ]`. La idea central es distinguir entre:

- **Carga incremental/exclusiva:** lo “nuevo” que entra antes de un sensor (diagnóstica para localizar puntos negros locales).
- **Carga acumulada:** todo lo que llega incluyendo aportes ya monitorizados aguas arriba (útil, pero menos específica para atribución local).

7.3.2 “Sensor mártir”: carga exclusiva con filtro de sensores intermedios

Para medir carga exclusiva por sensor, se buscan caminos desde cada vertido `v` hasta un sensor `s` siguiendo la red hidrológica, imponiendo que **no exista otro sensor entre medias** (es decir, `s` es el primer punto de control que intercepta ese vertido). En Cypher, el patrón crítico es el filtro `none(...)` sobre los nodos del camino:

```

MATCH (s:Sensor)
MATCH path = (v:Vertido)-[:AGUAS_ABAJO|
FLUYE_HACIA*1..100]->(s)
WHERE none(n IN nodes(path)[1..-1] WHERE n:Sensor)
WITH s, count(v) as num_vertidos_locales,
sum(v.volumen) as volumen_local_m3
WHERE volumen_local_m3 > 0
RETURN s.name as SENSOR, s.rio as RIO,
       num_vertidos_locales as NUEVOS_VERTIDOS_TRAMO,
       volumen_local_m3 as VOLUMEN_LOCAL_M3

```

Este enfoque es especialmente valioso para priorizar vigilancia y calibración: si un sensor “debería” ver variabilidad (por carga local alta) y aparece una señal plana o físicamente imposible, aumenta la probabilidad de fallo instrumental.

7.4 Baseline y calibración: definir “normalidad” antes de limpiar o alertar

7.4.1 Golden Batch: referencia local frente a medias globales

La detección de anomalías fiable requiere comparar contra un estado “normal” realista. En lugar de usar una media global del histórico (potencialmente sesgada por episodios de contaminación, extremos o fallos de sensórica), se define un baseline local mediante un **Golden Batch**: un tramo representativo de condiciones óptimas. Ejemplo operativo: para la estación **C333** en **noviembre de 2025**, se identificó un Golden Batch con valores típicos:

- pH $\approx$ 7.2
- Temperatura $\approx$ 16 °C

- Oxígeno ≈ 4.0 (estable)
- Amonio ≈ 0.0
- Nitratos ≈ 6.2

Este baseline se usa como patrón de calibración para decidir si un cambio observado es: (a) variación natural, (b) evento ambiental real, o (c) artefacto de adquisición/calibración. En visualizaciones exploratorias del tramo de referencia y su estabilidad, se documentan salidas

para respaldar la selección del periodo.

7.4.2 Normal vs. log-normal: tolerancias coherentes con la distribución

La calibración no solo fija niveles esperados, también define **cómo medir desviaciones** según la distribución de cada métrica:

- Aproximadamente normales (z-score directo más razonable):
- Cola pesada / log-normales (mejor en escala log o con umbrales asimétricos):

Esto evita errores comunes: en métricas log-normales, pequeñas variaciones absolutas cerca de cero pueden ser irrelevantes, mientras que incrementos multiplicativos sí lo son. Por tanto, el Golden Batch aporta el “centro” y la taxonomía estadística aporta la forma correcta de medir “distancia a lo normal”.

7.5 Inspección de anomalías: separar fallos físicos de eventos plausibles

7.5.1 Validación por límites físicos (HARD_LIMITS) y diagnóstico rápido

Antes de imputar o suavizar, se aplica una validación por “física estática” para detectar valores imposibles. Un hallazgo representativo aparece en **FICOCIANINAS**, donde se observan valores negativos pese a que el rango válido está definido como:

- **FICOCIANINAS** $\in [0.0, 1000.0]$

El filtrado y la descripción de outliers (en GPU, si se está usando RAPIDS) permite confirmar patrón y severidad:

```
ficos_neg = df_gpu[
    (df_gpu['metric'] == 'FICOCIANINAS') &
    (df_gpu['value'] < 0)
]
print(ficos_neg['value'].describe())
print(ficos_neg['value'].head(5))
```

Resultados relevantes:

count	28.000000
mean	-0.109567
std	0.022259
min	-0.223144
25%	-0.105361
50%	-0.105361
75%	-0.105361
max	-0.105361

La fuerte concentración en **-0.105361** sugiere un valor repetido típico de error de instrumentación/codificación (sensor roto, descalibrado u offline devolviendo un “código” numérico). Este tipo de señal suele coexistir con fallos tipo flatline (constancia prolongada) y debe tratarse como **calidad de dato**, no como fenómeno ambiental.

7.5.2 Contraste con comportamiento plausible: AMONIO dentro de rango

Para evitar “limpiar” eventos reales, se contrasta con métricas que muestran comportamiento esperado. En **AMONIO** (clasificada como log-normal y con límites físicos [0.0, 20.0]), los estadísticos observados son coherentes:

count	21781.000000
mean	0.886934
std	1.056485
min	0.000000
25%	0.048790
50%	0.392042
75%	1.350667
max	4.875197

No hay negativos y el máximo está lejos del límite, lo que sugiere variabilidad ambiental plausible más que fallo.

7.6 Segmentación temporal: rupturas por batch y silencios globales

7.6.1 Por qué segmentar antes de calcular volatilidad o imputar

Con sensores heterogéneos, la discontinuidad temporal introduce artefactos: si se calcula volatilidad sin separar huecos o cambios de ingestión, un salto temporal puede interpretarse como “alta variabilidad” cuando en realidad es un periodo sin datos. Por ello, se segmenta la serie multisensor en tramos “continuos” y comparables, detectando rupturas debidas a:

- **Cambios de lote** (`batch`), asociados a cambios de ingestión/procesamiento.
- **Silencios globales prolongados**, interpretados como caídas del sistema o huecos de adquisición.

El umbral típico para datos con cadencia horaria es **1.1 horas**: diferencias normales entre filas suelen ser ms/seg cuando varios sensores reportan alrededor de la misma hora; un salto > 1.1h suele indicar un hueco real.

7.6.2 Implementación vectorizada $O(n)$: diff/shift/cumsum

La técnica crítica evita bucles Python y funciona en una pasada si el DataFrame está ordenado por `timestamp` :

```
time_diff = df['timestamp'].diff()

cond_batch = df['batch'] != df['batch'].shift()
cond_gap = time_diff > pd.Timedelta(hours=1.1)

df['segment_id'] = (cond_batch | cond_gap).cumsum()
```

`segment_id` define “islas” temporales donde se asume continuidad operacional. Esta segmentación es la base para cálculos posteriores (deltas, volatilidad, detección de flatlines e imputación), porque impide cruzar periodos no comparables.

7.7 Deltas y volatilidad “propios”: medir dinámica por sensor dentro de cada segmento

7.7.1 Evitar falsos incrementos por densidad de reportes ajena

En datasets multisensor, la frecuencia de reporte de otros dispositivos puede distorsionar métricas si se comparan filas consecutivas globales. La solución es calcular deltas comparando cada sensor **consigo mismo**, dentro del mismo `segment_id` y por `metric`. La agrupación recomendada es:

- `segment_id` (no cruzar rupturas),
- `sensor_id` (comparación propia),
- `metric` (no mezclar magnitudes).

La técnica crítica usa `groupby().shift()` para obtener el valor y timestamp previos del mismo sensor:

```

g = df.groupby(['segment_id', 'sensor_id', 'metric'])

df['prev_value'] = g['value'].shift(1)
df['prev_timestamp'] = g['timestamp'].shift(1)

df['delta_abs'] = (df['value'] -
df['prev_value']).abs()
# df['own_time_diff'] = df['timestamp'] -
df['prev_timestamp'] # delta temporal propio

```

Con **delta_abs** y el delta temporal propio (cuando se calcula), la volatilidad deja de estar contaminada por “saltos” debidos a que un sensor dejó de reportar mientras otros siguieron. En combinación con el baseline (Golden Batch) y la clasificación normal/log-normal, estas métricas permiten definir umbrales más estables para anomalías y preparar la limpieza de fallos persistentes (como flatlines) sin borrar eventos reales.

7.8 Comparación breve de enfoques: proximidad vs. hidrología, baseline local vs. global

A lo largo del pipeline aparecen dos pares de estrategias complementarias:

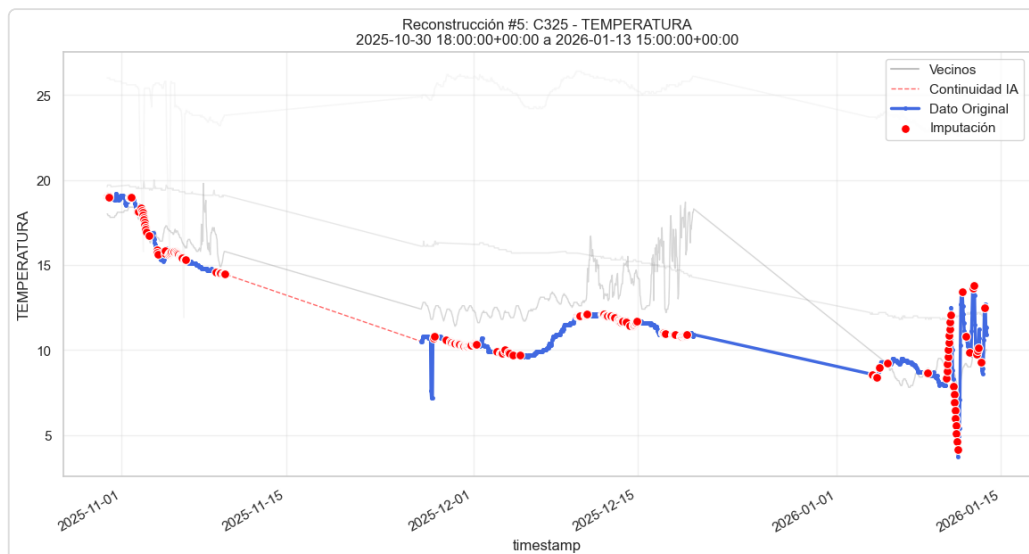
- **Asignación por proximidad (**ASIGNADO_TEORICO**) vs. relación hidrológica (**AGUAS_ABAJO**)**: la proximidad maximiza cobertura cuando falta semántica hidrológica; la hidrología aporta interpretabilidad causal. Mantener ambas permite análisis graduados (filtrar por “fuerza” de relación o ponderar).
- **Baseline local (Golden Batch) vs. media global**: el baseline local reduce sesgo por episodios extremos y fallos; la media

global puede servir como referencia amplia, pero es menos fiable para calibración y detección de cambios sutiles.

Integradas, estas decisiones convierten un conjunto de series ruidosas y desconectadas en un sistema analítico donde el dato temporal se interpreta con contexto espacial, y la limpieza se apoya en segmentación, física básica y estadística coherente con cada métrica.

8. Análisis y técnicas avanzadas

8.1 Análisis avanzado geoespacial e integración operativa



[Figura 10.]⁸

El objetivo de este bloque de técnicas avanzadas es convertir un inventario heterogéneo de **1835 nodos de vertido autorizados** y un

8. Reconstrucción de serie temporal de TEMPERATURA (30-10-2025 al 13-01-2026) mostrando los datos originales en azul, imputaciones como puntos rojos, una línea roja discontinua para la continuación de la IA, y un trazo tenue gris de referencia de Vecinos. La temperatura cae de ~19°C a finales de octubre a unos 9–12°C a mediados de enero, con densas imputaciones agrupadas a finales de diciembre y mediados de enero.

conjunto de **estaciones/sensores** en un sistema analítico coherente, donde sea posible (1) **priorizar** ejes fluviales relevantes, (2) **ordenar** elementos de forma consistente **aguas arriba → aguas abajo**, (3) **vincular** vertidos con sensores potencialmente afectados y (4) **hacer accionable** el resultado mediante una capa de atribución (responsables y jurisdicciones). La clave es pasar de etiquetas y geometrías dispersas a una referencia longitudinal común sobre el río y a relaciones reutilizables (tablas/grafos) para análisis posteriores.

8.1.1 Inventario de receptores y priorización de ejes principales

El punto de partida es reconocer que el campo **MEDIO_RECEPTOR** suele contener una mezcla de tipologías (río, arroyo, barranco, garganta, laguna) y categorías ambiguas o no fluviales (p. ej., **TERRENO**, “FILTRO VERDE”). Antes de cualquier modelado espacial fino, se realiza un **inventario de valores únicos** para evaluar diversidad, inconsistencias de nomenclatura y granularidad excesiva:

```
df['MEDIO_RECEPTOR'].unique()
```

Esta inspección tiene una implicación operativa directa: si se intenta analizar “tal cual” cada receptor, el esfuerzo se diluye en una larga cola de cursos menores con pocos vertidos. Por eso se priorizan los **ejes estructurantes** por frecuencia de descargas, destacando:

- **RÍO TAJO** — 55 vertidos
- **RÍO JARAMA** — 27 vertidos
- **RÍO ALBERCHE** — 21 vertidos

La lectura práctica es doble: (a) estos ríos concentran suficiente actividad como para justificar inspección y validación intensiva, y (b) sirven como “columna vertebral” para construir un eje

longitudinal (kilometraje fluvial) que permita integrar vertidos y sensores en un mismo marco. Aun así, el inventario también confirma que **muchos nodos vierten a tributarios o receptores secundarios**, por lo que el análisis posterior suele requerir una **agregación jerárquica** (tributario → río principal receptor final) para mantener una interpretación consistente.

8.2 Construcción de geometrías fluviales reproducibles (OSM/Overpass → UTM)

Una vez definidos los ríos prioritarios, el siguiente paso es disponer de una geometría de eje fluvial fiable sobre la que proyectar puntos. Para ello se descargan geometrías desde OpenStreetMap mediante Overpass, consultando entidades tipo `way` acotadas por:

- `waterway="river"`
- `name="{nombre_río}"`
- un bounding box aproximado (lat/lon) para limitar la búsqueda

La respuesta suele devolver múltiples tramos (segmentos) por río. Para convertirlos en un eje utilizable en cálculos de distancia y ordenación longitudinal, se reconstruyen `LineString` por tramo y se fusionan cuando es posible:

- Lista de `LineString` → `MultiLineString`
- Fusión con `linemerge(...)` para obtener una geometría más continua

8.2.1 Persistencia local y control de redundancia

Para evitar dependencia de red y repetir descargas (además de reducir el riesgo de rate limiting), se implementa un patrón de persistencia local por río:

- Directorio: `"/tmp/pycharm_project_992/data/datasets/raw/"`
- Archivo: `linestring_{safe_name}.csv` (nombre normalizado)

El flujo es determinista:

- Si existe el CSV, se carga con `pd.read_csv` y se rehidrata la geometría desde WKT (`wkt.loads`).
- Si no existe, se descarga, se construye la geometría y se guarda para ejecuciones futuras.

Este enfoque mejora la reproducibilidad del análisis y facilita depuración (se puede inspeccionar el WKT persistido si aparecen discontinuidades).

8.2.2 Reproyección a UTM para métricas en metros

Aunque OSM trabaja en `EPSG:4326` (lat/lon), las métricas que interesan (distancia lateral al cauce y distancia acumulada a lo largo del río) deben medirse en unidades lineales consistentes. Por ello se crea un `GeoDataFrame` en `EPSG:4326` y se reproyecta a `EPSG:32630` (UTM 30N), lo que permite:

- longitudes y distancias en **metros**
- proyecciones geométricas robustas para derivar “kilometraje fluvial”

8.2.3 Robustez operativa: reintentos y HTTP 429

Dado que Overpass puede responder con limitaciones, el proceso incorpora tolerancia a fallos:

- `max_retries = 5`
- manejo explícito de **HTTP 429**
- esperas incrementales (p. ej., `10 + intento5` segundos) y *backoff adicional
- `timeout` elevado (p. ej., `timeout=20`)

En conjunto, esto reduce fallos intermitentes y hace el pipeline más estable en ejecuciones repetidas.

8.3 Proyección hidráulica: de coordenadas UTM a eje longitudinal (km_rio) y control de calidad (dist_al_rio)

Con el eje del río en UTM, la técnica central es transformar cada punto (vertido o estación) desde coordenadas planas a dos métricas comparables sobre el cauce:

- `km_rio` : distancia acumulada desde el origen geométrico de la línea del río hasta la proyección ortogonal del punto sobre el eje. En UTM se obtiene en **metros** y puede expresarse como “kilómetro fluvial” con `km_rio / 1000` .
- `dist_al_rio` : distancia mínima (euclídea) del punto a la geometría del río. Funciona como **control de calidad**: valores altos sugieren mala georreferenciación o puntos fuera del tramo analizado.

La implementación se apoya en operaciones estándar de Shapely sobre una `LineString` :

- `geom.project(pt)` → distancia a lo largo del eje hasta la proyección

- `pt.distance(geom)` → distancia lateral mínima al eje

Código de referencia:

```
from shapely.geometry import Point
import pandas as pd

def get_hydro_metrics(row, geom, x_col, y_col):
    """
    Proyecta un punto (UTM) sobre la línea del río.
    Retorna:
        - km_rio: Distancia desde el nacimiento hasta el
        punto proyectado.
        - dist_al_rio: Distancia mínima desde el punto
        hasta el río.
    """
    pt = Point(row[x_col], row[y_col])
    return pd.Series(
        [geom.project(pt), pt.distance(geom)],
        index=['km_rio', 'dist_al_rio']
    )
```

8.3.1 Aplicación homogénea a vertidos y estaciones

La potencia de este enfoque es que se aplica igual a DataFrames distintos, generando columnas compatibles:

- En vertidos: a partir de `UTM_X`, `UTM_Y`
- En estaciones/sensores: a partir de `utm_x`, `utm_y` (o el esquema equivalente)

El resultado es un espacio de comparación común: ambos conjuntos quedan ordenables por `km_rio` y filtrables por `dist_al_rio`, habilitando análisis longitudinales y cruces con

series temporales (anomalías) sin depender de operaciones geométricas complejas en cada consulta.

8.4 Lógica de asignación por proximidad: “primer sensor aguas abajo” con umbral de ribera

Una vez que vertidos y estaciones comparten el mismo eje (`km_rio`), se puede construir una relación operativa clave: para cada vertido, identificar qué estación es la primera que podría registrar su señal aguas abajo. Esta lógica resuelve un problema típico de integración geoespacial-temporal: traducir posiciones en el mapa a una **topología consultable** (vertido → sensor) que luego se cruza con eventos/anomalías en series temporales.

8.4.1 Conceptos y regla de negocio

- **Umbral de ribera** (`UMBRAL_RIBERA`): distancia lateral máxima para considerar que un elemento pertenece al “corredor fluvial” del eje principal. En esta implementación: **3000 m (3 km)**.
- **Criterio aguas abajo**: para un vertido con `km_rio = k`, se buscan estaciones con `km_rio > k` y se selecciona la de menor `km_rio` (la inmediatamente posterior en el eje).

Este enfoque prioriza el cauce principal y reduce ruido de puntos alejados (p. ej., vertidos en tributarios mal alineados con el eje principal o estaciones fuera del corredor).

8.4.2 Implementación (filtrado + ordenación + asignación)

El patrón computacional es simple y determinista:

- Filtrar vertidos y estaciones por `dist_al_rio < UMBRAL_RIBERA`
- Ordenar estaciones por `km_rio`
- Para cada vertido, seleccionar la primera estación con `km_rio` superior y guardar la relación con distancia longitudinal

Fragmento crítico:

```

UMBRALES_RIBERA = 3000
df_rio = df[df['dist_al_rio'] < UMBRALES_RIBERA].copy()
stations_rio = stations[stations['dist_al_rio'] <
UMBRALES_RIBERA].copy()

stations_rio = stations_rio.sort_values('km_rio')

rels_hidro = []

for idx, vertido in df_rio.iterrows():
    # buscar estaciones aguas abajo (km_rio mayor)
    candidates = stations_rio[stations_rio['km_rio'] >
vertido['km_rio']]
    if len(candidates) == 0:
        continue
    sensor = candidates.iloc[0]
    rels_hidro.append({
        'source_id': vertido['REF_EXPEDIENTE'],    # o
id del vertido
        'target_id': sensor['station_id'],        # id
de estación
        'tipo_relacion': 'AGUAS_ABAJO',
        'dist_long_m': sensor['km_rio'] -
vertido['km_rio']
    })

```

Comparación breve de enfoques: frente a asignar por “estación más cercana en distancia euclídea”, esta estrategia es más coherente hidráulicamente para ríos principales porque respeta la direccionalidad (aguas abajo) y usa una métrica longitudinal estable. Su limitación es que asume que el eje del río representa bien la conectividad real; en redes muy ramificadas o con tributarios relevantes, conviene complementar con jerarquía hidrográfica (tributario → principal) o modelos de red más explícitos.

8.5 Capa de atribución: del grafo físico al grafo accionable (titular y municipio)

Con la capa espacial (posición/proyección) y la capa hidráulica (relaciones aguas abajo), el sistema ya permite análisis de propagación y priorización técnica. Sin embargo, para cerrar el ciclo operativo (inspección, trazabilidad, coordinación institucional) se añade una **Attribution Layer** que responde a: **quién** es responsable y **dónde** aplica la jurisdicción.

8.5.1 Modelo de relaciones (expediente como nodo ancla)

El patrón recomendado es usar `REF_EXPEDIENTE` como nodo central y crear relaciones hacia entidades normalizadas:

- `(Expediente) -[:TIENE_TITULAR]->(Titular)`
- `(Expediente) -[:EN_MUNICIPIO]->(Municipio)`

En el material original aparecen variantes de etiquetas (p. ej., `responsabilidad / jurisdiccion`). Para evitar duplicidades semánticas en consultas y en el pipeline, conviene **estandarizar** nombres (por ejemplo, mantener `TIENE_TITULAR` y `EN_MUNICIPIO`) y, si se requiere, añadir un campo `tipo_relacion` como propiedad.

8.5.2 Normalización crítica para evitar duplicados

Al construir nodos desde texto libre, el principal riesgo es fragmentar el grafo por variaciones de formato (espacios, mayúsculas/minúsculas). La técnica mínima efectiva es normalizar con `strip()` + `upper()` antes de generar relaciones:

```

# Normalizamos el nombre del titular para evitar
duplicados por espacios o mayúsculas
df['TITULAR_NORM'] =
df['TITULAR'].str.strip().str.upper()

# Relación expediente -> titular (responsabilidad)
rel_titular = df[['REF_EXPEDIENTE',
'TITULAR_NORM']].copy()
rel_titular.columns = ['source_id', 'target_id']
rel_titular['tipo_relacion'] = 'TIENE_TITULAR'

# Normalizamos el nombre del municipio para evitar
duplicados por espacios o mayúsculas
df['MUNICIPIO_NORM'] =
df['MUNICIPIO'].str.strip().str.upper()

# Relación expediente -> municipio (jurisdicción)
rel_municipio = df[['REF_EXPEDIENTE',
'MUNICIPIO_NORM']].copy()
rel_municipio.columns = ['source_id', 'target_id']
rel_municipio['tipo_relacion'] = 'EN_MUNICIPIO'

```

El efecto práctico es inmediato: mejores agregaciones (“titulares con más expedientes”, “municipios con mayor densidad de vertidos”), menos duplicados y consultas más fiables cuando se combine con la capa hidráulica (por ejemplo, “titulares con vertidos cuyo primer sensor aguas abajo muestra anomalías recurrentes”).

8.6 Cierre: de geometría y texto libre a relaciones reutilizables

Estas técnicas avanzadas forman una cadena coherente: (1) se priorizan ejes principales a partir del inventario de receptores, (2) se construyen geometrías fluviales reproducibles y métricas en UTM, (3) se proyectan puntos a un eje longitudinal (`km_rio`) con control de calidad (`dist_al_rio`), (4) se deriva una relación hidráulica simple pero operativa (“primer sensor aguas abajo” bajo umbral de ribera) y (5) se completa el grafo con atribución (titular/municipio) mediante normalización estricta. El resultado es una base sólida para análisis posteriores de irregularidades, correlación con anomalías en sensores y priorización de actuaciones con responsables claramente identificados.

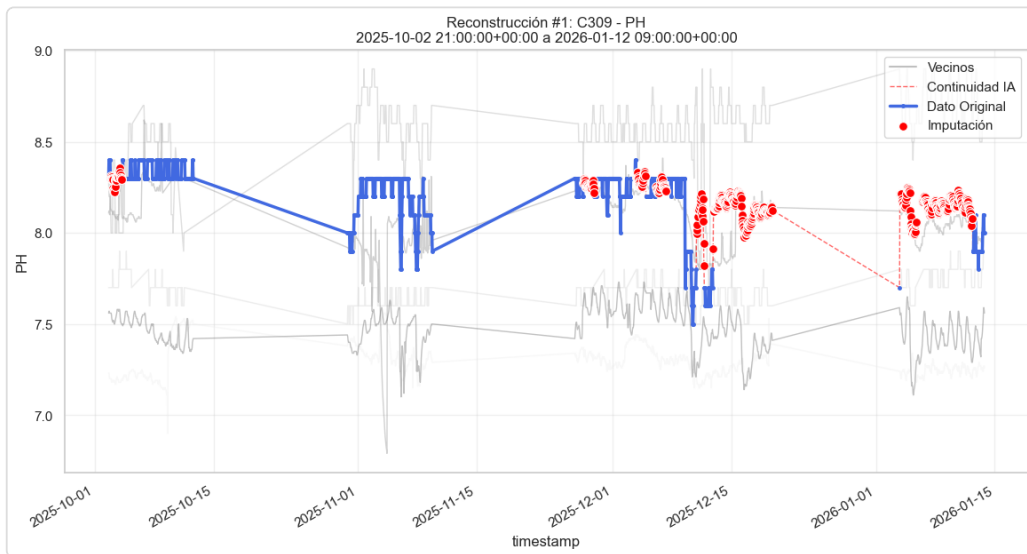
9. Calidad y aseguramiento de datos

9.1 Alcance de “Data Quality and Assurance” en series ambientales

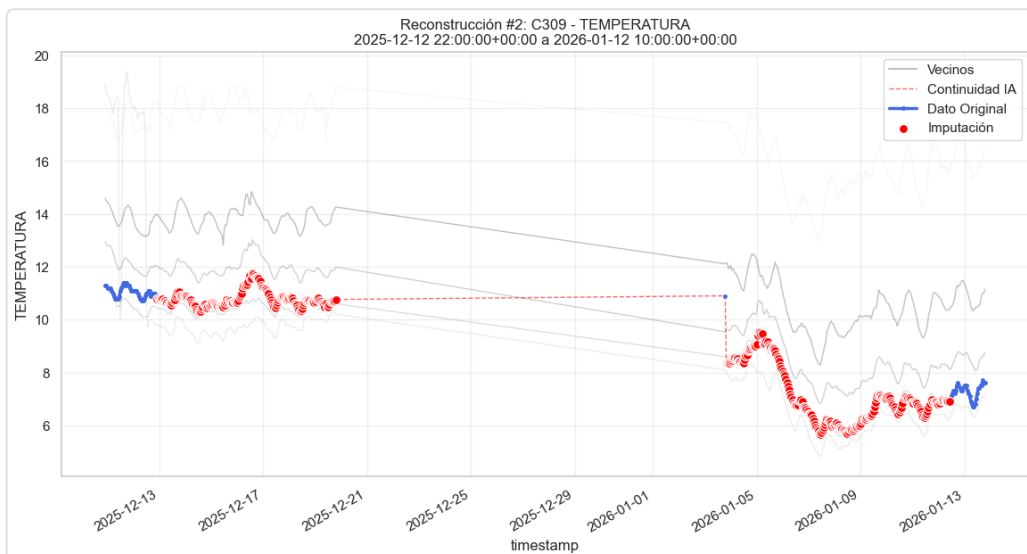
El aseguramiento de calidad (QA/QC) en este proyecto se diseña para que las series temporales ambientales (temperatura, oxígeno disuelto, pH, clorofila/ficocianinas, conductividad, turbidez, nutrientes, etc.) sean **físicamente plausibles**, **temporalmente coherentes** y **analíticamente útiles** antes de cualquier etapa sensible al contexto: detección de anomalías, imputación avanzada o reparación de fallos como flatlines (tramos anormalmente constantes). La lógica general es secuencial y conservadora:

- **Eliminar imposibles** (hard limits) para que no contaminen estadísticas y modelos.
- **Aplicar coherencia temporal** (física dinámica) para filtrar “glitches” incompatibles con la inercia del sistema.
- **Depurar outliers extremos con contexto local** (Rolling Z-Score), evitando sesgos por cambios de régimen.
- **Imputar huecos** con estrategias diferenciadas (desde “smart infilling” conservador hasta modelos multivariantes), siempre **por segmentos/batches** para no mezclar periodos no comparables.

Cuando se documentan resultados exploratorios o mapas/series, se referencian salidas visuales de cada parte con el formato:



[Figura 11.] Reconstrucción de pH en C309



[Figura 12.]⁹

9. Gráfico lineal de temperatura a lo largo del tiempo (12-12-2025 al 12-01-2026). El azul indica los datos originales, los puntos rojos y la línea discontinua roja muestran imputaciones y continuidad de la IA, mientras que las líneas grises representan reconstrucciones vecinas; la temperatura cae de unos 11°C a finales de diciembre a unos 6–7°C a principios de enero, con valores imputados rellenos los huecos.

9.2 Reglas de dominio: variables con ciclo diario y señales de fallo

9.2.1 Comportamiento esperado en entornos acuáticos

Varias métricas clave presentan **ciclo diurno**: su variación día–noche no es ruido, sino parte de la señal ambiental. Por tanto, la calidad no se evalúa solo por “estar dentro de rango”, sino por **consistencia con la dinámica esperable**.

- **Temperatura**: oscila día/noche; la **inercia térmica** impide cambios instantáneos, pero también hace improbable una estabilidad perfecta (siempre hay pequeñas variaciones decimales). Picos aislados pueden ser outliers o eventos reales y deben interpretarse con contexto temporal.
- **Oxígeno disuelto (OD)**: suele subir de día (fotosíntesis) y bajar de noche (respiración). Un OD “plano” durante horas es típico de membrana sucia, sensor colgado o fallo de lectura.
- **pH**: en ríos suele fluctuar con OD por el equilibrio CO_2 /carbonatos. Un pH clavado (p. ej., 7.00 u 8.15) durante varias horas es sospechoso salvo condiciones muy tamponadas.
- **Clorofila / Ficocianinas**: responden a luz, mezcla y corrientes; largos periodos constantes sugieren **biofouling** (algas adheridas) y el sensor “leyendo su propia suciedad”.

Estas reglas de dominio justifican por qué el pipeline **limpia primero** (imposibles y glitches) y **luego** aplica detectores e imputadores: si se dejan errores extremos o valores físicamente imposibles, se distorsionan medias/desviaciones locales y se degradan los pasos posteriores (incluida la detección de flatlines).

9.3 QC por física estática y dinámica: el “suelo” mínimo de calidad

9.3.1 Hard limits: plausibilidad físico-química

La política base es estricta: cualquier valor fuera de rango se considera error crítico y se convierte a **NaN**. Esto protege tanto la estadística móvil (p. ej., Rolling Z-Score) como la imputación multivariante (evita que el modelo “aprenda” desde valores imposibles). Ejemplos de límites utilizados (coherentes con la configuración del flatline cleaner):

- **AMONIO:** 0.0–20.0 mg/L
- **PH:** 4.0–10.5
- **CONDUCTIVIDAD:** 20.0–5000.0 $\mu\text{S}/\text{cm}$
- **TEMPERATURA:** 0.001–38.0 °C (evitando ceros exactos, patrón típico de fallo)
- **OXÍGENO DISUELTO:** 0.001–20.0 mg/L
- **TURBIDEZ:** 0.0–2000.0 NTU
- **CARBONO ORGÁNICO:** 0.0–50.0 mg/L (con política adicional de “no cero exacto” cuando aplique)

En términos de QA, esta capa es “no negociable”: un valor imposible no se corrige con suavizados; se **anula** para impedir contaminación aguas abajo.

9.3.2 Coherencia temporal: física dinámica (ejemplo explícito)

Además de rangos, se aplican reglas de consistencia entre muestras consecutivas, especialmente en variables de alta inercia. Caso documentado:

- **Temperatura:** si el salto entre registros consecutivos cumple

La motivación es física: en agua, un cambio tan brusco en una hora (o intervalo equivalente) es implausible y suele indicar glitch, desconexión o lectura corrupta. Esta regla reduce falsos “eventos” que luego podrían confundirse con anomalías reales.

9.4 Outliers con contexto: Rolling Z-Score (y por qué no basta el z-score global)

9.4.1 Distribuciones típicas: normales vs. cola pesada

El filtrado de outliers se apoya en un análisis previo de distribución, porque no todas las métricas se comportan igual:

- **Aproximadamente normales** (colas moderadas): `PH`, `TEMPERATURA`, `OXIGENO DISUELTO`, `NIVEL`.
- **Log-normales / cola pesada** (picos raros grandes; media $\gg$ mediana): `AMONIO`, `TURBIDEZ`, `NITRATOS`, `FOSFATOS`, `CLOROFILA`, `FICOCIANINAS`, `CONDUCTIVIDAD`, `CARBONO ORGANICO`.

Implicación práctica: un z-score “directo” sobre métricas de cola pesada tiende a marcar como outlier eventos plausibles (p. ej., picos tras lluvia). Por ello se aplica una **transformación logarítmica suave** (`log1p`) antes de calcular el z-score en ese grupo.

9.4.2 Rolling Z-Score: estadística local y segmentación por batch

En lugar de un z-score global (que mezcla estaciones, cambios de régimen y estacionalidad), se usa **z-score en ventana móvil** para comparar cada punto con su **contexto cercano**. Además, el cálculo se hace por `sensor_id` y `metric`, y se reinicia por `batch` para no contaminar medias/desviaciones entre periodos desconectados

(cortes de datos, recalibraciones, paradas de estación, etc.). Parámetros documentados (criterio conservador para purgar errores extremos sin borrar eventos ambientales):

- `window = 96` (≈ 4 días si la frecuencia es horaria): captura variabilidad meteorológica sin sobre-reaccionar.
- `min_periods = 24` (≈ 1 día): evita decisiones sin contexto suficiente.
- `sigma = 6.0` : umbral alto (conservador) para eliminar extremos claros.
- `epsilon = 1e-6` : evita división por cero cuando la desviación local es casi nula (situación que, además, puede ser indicio de flatline).

Código clave (selección de métricas y parámetros):

```

METRICS_NORMAL = ['PH', 'TEMPERATURA', 'OXIGENO
DISUELTO', 'NIVEL']
METRICS_LOG = ['AMONIO', 'TURBIDEZ', 'NITRATOS',
'FOSFATOS',
                'CLOROFILA', 'FICOCIANINAS',
'CONDUCTIVIDAD', 'CARBONO ORGANICO']

ZSCORE_PARAMS = {
    'window': 96,          # 4 días (96 horas)
    'min_periods': 24,     # mínimo 1 día
    'sigma': 6.0,          # umbral de corte
    'epsilon': 1e-6        # evitar división por cero
}

def apply_zscore_outliers(df):
    """
    Rolling Z-Score por sensor/métrica/batch.
    - loglp para métricas de cola pesada.
    - rolling mean/std en ventana móvil.
    - marca outliers (|z| > sigma).
    """
    ...

```

En la práctica, este enfoque “rolling + batch” es el puente entre QC físico (hard limits/coherencia temporal) y las etapas de reparación: elimina extremos que distorsionarían tanto la imputación como la detección de fallos instrumentales.

).

9.5 Imputación como parte del QA: conservadora primero, avanzada después

9.5.1 Principio de orden: limpiar antes de imputar, e imputar antes de reconstrucciones sensibles

Los huecos degradan el análisis físico-químico y también técnicas basadas en contexto (como Rolling Z-Score o detectores de flatlines). Por eso, la imputación se ubica **después de la limpieza básica** (hard limits, deduplicación, coherencia temporal) y **antes de procedimientos más sensibles** (outliers multivariantes, reconstrucción de tramos largos, reparación específica de fallos). Un requisito transversal es **no mezclar periodos desconectados**: tanto el cálculo de outliers como la imputación se aplican **por batch/segmento**, tratando cada tramo continuo como un contexto independiente.

9.5.2 Smart infilling (diferenciado por tipo de variable)

La primera capa de imputación es deliberadamente prudente: **rellenar solo huecos pequeños** y evitar “inventar” eventos o borrar ciclos día/noche. La política se adapta a la naturaleza de la métrica:

- **Grupo A — Alta inercia (“lentos”):** **TEMPERATURA** , **NIVEL** , **CONDUCTIVIDAD**
- Método: interpolación temporal lineal (**time/linear**)
- Gap máximo: 6 horas
- Justificación: cambios suaves; en ventanas cortas el sesgo es bajo.
- **Grupo B — Biológicas/cíclicas (“ritmos circadianos”):** **OXÍGENO DISUELTO** , **CLOROFILA** , **FICOCIANINAS** , **PH**
- Método: se evita una imputación agresiva que aplane máximos/mínimos; se prioriza conservar la forma del ciclo y

dejar **NaN** cuando el riesgo de distorsión es alto (especialmente en huecos largos o en presencia de fallos instrumentales).

Esta capa “smart” contrasta con imputaciones genéricas (p. ej., forward fill) que pueden perpetuar constantes y enmascarar flatlines.

9.5.3 Imputación univariante vs. multivariante: comparación breve

- **Univariante (baseline):** interpolación, forward/backward fill, medias móviles. Es rápida y útil para huecos pequeños, pero puede **aplanar** ciclos diurnos y fallar en cambios rápidos reales (tormentas, descargas, mezcla). Además, si hay flatlines, rellenar con constantes puede empeorar el problema.
- **Multivariante (recomendada cuando hay correlación):** usa variables/sensores correlacionados para reconstruir el valor faltante. En redes acuáticas, la **temperatura** suele tener alta multicolinealidad espacial, lo que habilita imputaciones estables. (Muñoz Rosas & Álvarez-Verdejo, 2009)

9.5.4 Imputación espacial guiada por topología con Ridge / Bayesian Ridge

La técnica avanzada principal predice el valor de un sensor objetivo usando **sensores vecinos** definidos por una topología (proximidad, conectividad hidráulica o correlación histórica). Se documenta un ejemplo reconstruyendo **TEMPERATURA** del sensor **C302** a partir de vecinos. Flujo (por métrica y por batch):

- **Ridge:** rápido, robusto ante multicolinealidad.
- **Bayesian Ridge:** alternativa probabilística, útil con pocos datos o ruido; puede aportar estabilidad e incertidumbre (según implementación).

Este enfoque se integra naturalmente con QA: al apoyarse en estructura espacial/topológica, reconstruye huecos sin destruir el ciclo diurno y sin introducir discontinuidades artificiales entre periodos no comparables. (Muñoz Rosas & Álvarez-Verdejo, 2009)

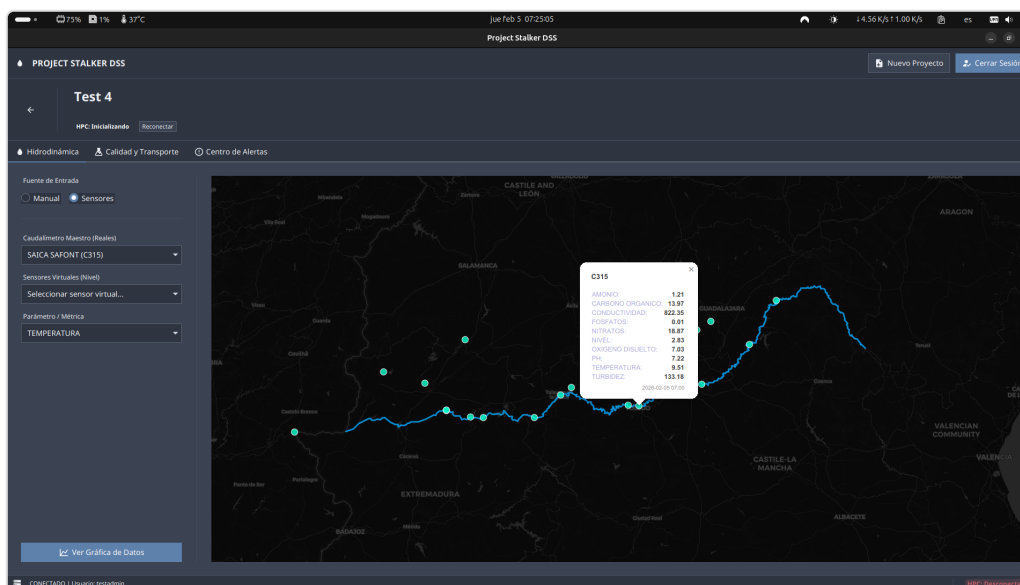
10. Conclusiones y elaboración de informes

10.1 Conclusiones del flujo integrado (calidad, continuidad y explotabilidad)

El flujo completo —análisis geoespacial de autorizaciones de vertido, depuración/normalización de datos y tratamiento de series temporales de sensores— culmina en un conjunto de datos **cohesivo, consistente y listo para analítica**. El valor principal del proceso no es únicamente “limpiar”, sino **convertir datos heterogéneos y potencialmente engañosos en un activo reutilizable** para etapas posteriores: análisis exploratorio avanzado, modelado de riesgo, detección de anomalías, auditoría y toma de decisiones operativas. En términos prácticos, el resultado se refleja en dos mejoras transversales:

- **Fiabilidad:** reducción de artefactos (especialmente fallos de instrumentación en sensores) que distorsionan métricas y modelos.
- **Precisión analítica:** menor sesgo inducido por errores sistemáticos (p. ej., señales planas que simulan estabilidad) y mayor coherencia temporal para análisis de tendencias, estacionalidad y alarmística.

En la siguiente imagen, se puede observar como se combina el flujo del río con la serie de sensores en la aplicación de escritorio diseñada para la monitorización del río



Mapa renderizado del río con los sensores en tiempo real

10.2 Gestión de anomalías en sensores: de la detección a la corrección

10.2.1 Detección y neutralización de “flatlines” (señales planas)

Uno de los degradadores más relevantes en series temporales ambientales es la presencia de **tramos prolongados de valores constantes anómalos** (flatlines), típicamente asociados a congelación de lecturas, bloqueos de instrumentación o fallos de transmisión/registro. El enfoque aplicado prioriza robustez frente a configuraciones rígidas:

- Los segmentos se **identifican sin depender de una ventana fija**, utilizando una lógica de acumulación de puntos de

sospecha que se adapta a distintas frecuencias de muestreo y duraciones del fallo.

- Una vez detectados, los tramos se **neutralizan convirtiéndolos en NaN**, evitando que valores constantes:
 - distorsionen estadísticas (media, varianza, percentiles),
 - oculten eventos reales (picos o cambios abruptos),
 - contaminen modelos de anomalías (aprendiendo una “normalidad” falsa basada en estabilidad artificial).

Esta decisión (flatline → NaN) es clave: transforma un fallo silencioso en **ausencia explícita de dato**, lo que mejora trazabilidad y reduce conclusiones erróneas en etapas posteriores.

10.2.2 Imputación avanzada e iterativa para restaurar continuidad

Tras eliminar artefactos (marcando periodos inválidos como NaN), el objetivo pasa de “quitar errores” a **recuperar continuidad temporal** sin introducir reconstrucciones irreales. Por ello se aplican técnicas de **imputación avanzada**, con dos principios:

- **Preservar coherencia temporal**: mantener dinámica y transiciones plausibles en la serie.
- **Respetar relaciones entre variables** cuando existen múltiples señales correlacionadas, evitando imputaciones aisladas que rompen consistencia multivariante.

Este enfoque se diferencia explícitamente de alternativas simplistas que suelen degradar el dataset:

- eliminar filas (fragmenta series y reduce cobertura),
- rellenar con ceros/constantes (introduce sesgo y “planicies” artificiales),
- imputaciones triviales que no capturan dinámica temporal.

Además, se realizan iteraciones sobre **hiperparámetros** para ajustar la calidad de la imputación, priorizando consistencia y

realismo frente a “rellenar por rellenar”. Por ejemplo, se ha determinado que el umbral de corte óptimo es un score combinado de 20

```
THRESHOLD_SCORE = 20.0

# Propagamos el veredicto:
# Calculamos el Score MÁXIMO que alcanzó cada
bloque y se lo pegamos a todas sus filas.
df_scored['max_score_in_block'] =
df_scored.groupby('block_id')
['stagnation_score'].transform('max')
```

10.2.3 Control de calidad final del entregable temporal

El proceso culmina en un dataset consolidado (referido durante el flujo como `df_final`) sometido a un control de calidad final orientado a consumo analítico:

- Se **eliminan registros remanentes no imputables**, especialmente aquellos en los que la columna `value` no puede recuperarse de forma satisfactoria.
- Se garantiza que el entregable no contenga puntos que puedan **romper análisis downstream** o introducir incertidumbre no controlada.

El resultado es un conjunto temporal donde los periodos inválidos quedan **corregidos o, si procede, excluidos**, reduciendo falsos positivos/negativos en detección de anomalías y estabilizando métricas de referencia.

10.3 Persistencia y escalabilidad: del dataset depurado a artefactos reutilizables

10.3.1 Serialización eficiente (Parquet) como “staging” operativo

Para que el dataset depurado sea explotable en escenarios de volumen (sensores de alta frecuencia, múltiples estaciones, actualizaciones periódicas), se consolida un enfoque de **persistencia por artefactos**, con **Parquet** como formato preferente por:

- lectura parcial por columnas,
- compresión eficiente,
- interoperabilidad con herramientas analíticas y de ingeniería de datos,
- trazabilidad del preprocesado (versionado de salidas y reproducibilidad).

Cuando el volumen lo requiere, el pipeline se apoya en un **preprocesado de alto rendimiento** (incluida aceleración GPU cuando está disponible) para transformar, validar y exportar grandes datasets sin cuellos de botella. En reporting técnico, conviene documentar qué artefactos se generan (p. ej., rutas, particiones, esquema) y qué etapa los produce, de modo que el consumo downstream (dashboards, auditoría, modelos) sea directo.

10.3.2 Dos niveles de producto: dataset completo vs subconjunto accionable

Una conclusión operativa importante es separar explícitamente:

- el **dataset completo**, útil para auditoría, análisis exploratorio, consistencia estadística y trazabilidad; y
- un **subconjunto crítico “accionable”**, diseñado para priorización, inspección y reporting ejecutivo.

Este patrón evita que decisiones operativas dependan de consultas complejas sobre el total, y al mismo tiempo preserva un repositorio íntegro para revisiones y explicabilidad.

10.4 Reporting orientado a decisión: “Club TIER 1” y métricas de concentración

10.4.1 Persistencia del subconjunto crítico (“Club TIER 1”)

En el bloque geoespacial, a partir del DataFrame jerarquizado `df_jerarquia`, se construye un DataFrame específico `df_oligarcas` que materializa el subconjunto de mayor impacto:

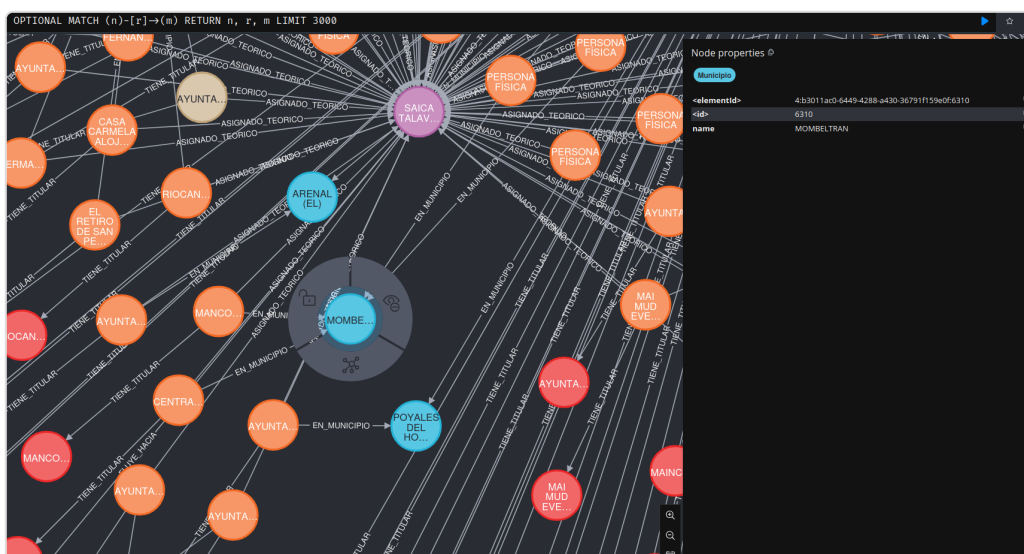
- filtra registros `TIER_1_CRITICO` en la columna `TIER`,
- ordena por `VOLUMEN_m3_ano` descendente para priorizar por impacto,
- queda listo para persistirse como tabla de trabajo, Parquet o insumo de cuadros de mando.

Este subconjunto es el puente natural entre analítica y operación: permite focalizar inspecciones o seguimiento en los actores con mayor contribución al volumen total.

10.4.2 Métricas y ranking: cuantificar impacto y priorizar acciones

El reporte de inteligencia se centra en medir **concentración del volumen** y producir un ranking operativo:

- total de expedientes analizados: `len(df_jerarquia)`
- número de titulares críticos: `len(df_oligarcas)`
- cuota de poder (share del volumen total):
- top de titulares con contexto: `TITULAR`, `MUNICIPIO`, `VOLUMEN_m3_ano`, `MEDIO_RECEPTOR`, `NATURALEZA_VERTIDO`, `tipo_titular`
- distribución por naturaleza del vertido dentro de TIER 1 (`value_counts()`), útil para segmentar acciones (urbano vs industrial).



Grafo centrado en un nodo Municipio

10.4.3 Código mínimo reproducible (extracción y cuota de poder)

```
df_oligarcas = (
    df_jerarquia[df_jerarquia['TIER'] ==
    'TIER_1_CRITICO']
    .sort_values('VOLUMEN_m3_ano', ascending=False)
)

total_volumen_cuenca =
df_jerarquia['VOLUMEN_m3_ano'].sum()
volumen_oligarcas =
df_oligarcas['VOLUMEN_m3_ano'].sum()
cuota_poder = (volumen_oligarcas /
total_volumen_cuenca) * 100
```

Este bloque resume el mecanismo de reporting: **seleccionar lo crítico, ordenar por impacto y cuantificar concentración**, dejando el resultado listo para persistencia y consumo por equipos de inspección, analítica o gobierno del dato.

10.5 Cierre final

La ejecución de este proyecto ha permitido transformar un conjunto de datos heterogéneo y ruidoso —compuesto por lecturas de sensores, registros administrativos y datos geoespaciales— en un activo estratégico para la gestión hidrológica. Más allá de la limpieza técnica, el verdadero valor reside en haber establecido un flujo de trabajo (pipeline) reproducible y auditable que garantiza que las decisiones se tomen sobre evidencia confiable y no sobre artefactos digitales.

La implementación de técnicas avanzadas, como la detección multinivel de flatlines y la aceleración por GPU, ha demostrado que

es posible mantener la integridad del dato incluso ante volúmenes masivos de información[cite: 20, 49]. Este enfoque no solo "arregla" los datos para el presente informe, sino que sienta las bases para un sistema de monitoreo continuo, donde la calidad del dato es proactiva y no reactiva.

Nota sobre la escalabilidad: La arquitectura modular basada en Parquet y el procesamiento vectorizado asegura que el sistema pueda absorber nuevos lotes de sensores o históricos extendidos sin requerir una refactorización del código base, garantizando la sostenibilidad técnica del proyecto a largo plazo.

Como resultado final, hemos transitado de una visión fragmentada a una inteligencia operativa consolidada. Las métricas de concentración (Ranking TIER 1) y los análisis de desigualdad (Gini/Lorenz) ofrecen ahora una hoja de ruta clara para priorizar intervenciones, enfocando los recursos de inspección y control en aquellos actores que representan la mayor cuota de impacto en el ecosistema.

Confianza en el Dato

La eliminación sistemática de anomalías silenciosas (como sensores atascados o flatlines) restaura la validez estadística de las series temporales, eliminando sesgos en los modelos predictivos posteriores.

Accionabilidad

El paso de coordenadas puras a relaciones topológicas (sensor-vertido) convierte mapas estáticos en grafos de responsabilidad, permitiendo atribuir cargas contaminantes a entidades específicas.

En conclusión, este reporte entrega no solo un diagnóstico del estado actual de la cuenca, sino las herramientas metodológicas y el código necesario para replicar este análisis, democratizando el acceso a una fiscalización ambiental rigurosa y basada en datos de alta precisión.

11. Referencias bibliográficas

- Alija, A. (2021). ¿Por qué deberías de usar ficheros Parquet si procesas muchos datos? [datos.gob.es](https://datos.gob.es/es/blog/por-que-deberias-de-usar-ficheros-parquet-si-procesas-muchos-datos).
<https://datos.gob.es/es/blog/por-que-deberias-de-usar-ficheros-parquet-si-procesas-muchos-datos>
- Amat Rodrigo, J. (2020). Detección de anomalías: Isolation Forest. *Cienciadedatos*. https://cienciadedatos.net/documentos/66_deteccion_anomalias_isolationforest
- Cabanillas Barbacil, D. (s. f.). Calidad de datos. Universitat Oberta de Catalunya. <https://openaccess.uoc.edu/server/api/core/bitstreams/2078adf2-3f80-4073-94c8-a4d1b4ad26ee/content>
- España. (2001). Real Decreto Legislativo 1/2001, de 20 de julio, por el que se aprueba el texto refundido de la Ley de Aguas. *Boletín Oficial del Estado*. <https://www.boe.es/eli/es/rdlg/2001/07/20/1/con>
- Kumar, R. (2026). Guía sobre el algoritmo de agrupamiento DBSCAN. *DataCamp*. <https://www.datacamp.com/es/tutorial/dbscan-clustering-algorithm>
- Lu, L., Jin, P., Pang, G., Zhang, Z., & Karniadakis, G. E. (2021). Learning nonlinear operators via DeepONet based on the universal approximation theorem of operators. *Nature Machine Intelligence*, 3, 218-229. <https://doi.org/10.1038/s42256-021-00302-5>
- Muñoz Rosas, J. F., & Álvarez-Verdejo, E. (2009). Métodos de imputación para el tratamiento de datos faltantes: aplicación mediante R/Splus. *Revista de métodos cuantitativos para la*

- economía y la empresa, 7, 3-30. <https://doi.org/10.46661/revmetodoscuanteconempresa.2120>
- Pérez Juste, R. (2012). Estadística aplicada a las Ciencias Sociales: Curso destinado a la preparación de futuros estudiantes de las asignaturas de Estadística de grados universitarios en CC. Sociales. Universidad Nacional de Educación a Distancia (UNED). https://www.intecca.uned.es/upload/noticias/20130614114556u_CURSO0DEESTADISTICAAPLICADA.pdf
- Soriano, P. (2024). Explorando el método DBSCAN: Análisis de agrupamiento por densidad con Scikit-learn. Geoinnova. <https://geoinnova.org/blog-territorio/explorando-el-metodo-dbscan-analisis-de-agrupamiento-por-densidad-con-scikit-learn/>
- Vietri, S., & Del Duca, S. (2023). Curva de Lorenz. Revista de Investigación en Modelos Matemáticos Aplicados a la Gestión y la Economía. Facultad de Ciencias Económicas, Universidad de Buenos Aires. <https://www.economicas.uba.ar/wp-content/uploads/2016/04/Curva-de-Lorenz-Vietri-Del-Duca-.pdf>
- Yepes Piqueras, V. (2022). ¿Qué hacemos con los valores atípicos (outliers)? El blog de Víctor Yepes (Universitat Politècnica de València). <https://victoryepes.blogs.upv.es/2022/02/21/que-hacemos-con-los-valores-atipicos-outliers/>